

PROGRAMMATION ORIENTÉE OBJET EN C++

Joel FALCOU



À propos de ce cours

Objectifs

- Comprendre ce qu'est réellement un objet en C++
- Maîtriser la durée de vie des objets et la sémantique de valeur
- Faire du **RAII** un réflexe et non une technique exotique
- Concevoir des hiérarchies et des interfaces qui vieillissent bien

Pré-requis

- Types, variables, fonctions, surcharge
- Structures agrégées, `std::vector`, `std::string`
- Fonctions génériques élémentaires, lambdas

Hors périmètre

- Un cours de génie logiciel abstrait
- Un catalogue de patrons à appliquer mécaniquement

Plan du cours

La classe et l'héritage

1. Des agrégats aux classes
2. Constructeurs et destructeurs
3. Héritage et polymorphisme dynamique

Valeurs et ressources

4. Sémantique de valeur, copie et déplacement
5. Fonctions membres spéciales
6. RAII et sûreté face aux exceptions

Conception

7. Principes de conception et SOLID
8. Patrons de conception
9. Au-delà du polymorphisme dynamique

1. Des agrégats aux classes

Modèle agrégat

Définition

- Un agrégat regroupe des membres de types et de noms arbitraires
- L'agrégat est le moyen fondamental d'ajouter un type au langage

```
1 struct mon_type
2 {
3     type_membre1 membre_1;
4     type_membre2 membre_2;
5     // autant que nécessaire
6     type_membreN membre_N;
7 };
```

Propriétés

- Tous les membres sont **publics**
- L'initialisation se fait par liste, membre à membre
- Aucune cohérence n'est garantie entre les membres

Initialiser un agrégat

Syntaxe

- `date d1;` laisse des **valeurs indéterminées** : lire `d1.jour` est indéfini
- `date d2{};` initialise par défaut : tous les membres valent zéro
- Un désignateur peut être omis, jamais déplacé

```
1  struct date { int jour; int mois; int annee; };
2
3  int main()
4  {
5      date d1;           // membres NON initialises
6      date d2{};         // membres initialises a zero
7      date d3{25, 12, 2024}; // initialisation par liste
8      date d4{.jour = 25, .mois = 12, .annee = 2024}; // designateurs, C++20
9      date d5{.jour = 25, .annee = 2024};           // mois saute, vaut 0
10 }
```

LIMITATIONS

Un agrégat ne valide aucune valeur : `date{.mois = 13}` est accepté, alors que treize n'est pas un mois.

Consommer un agrégat

Principe

- Une liaison structurée nomme les champs au lieu de les indexer
- Il faut **autant de noms que de membres**
- Elle ne s'applique qu'aux agrégats, pair et tuple

```
1  date p{25, 12, 2024};
2
3  auto [jour, mois, annee] = p;   // liaisons structurees, C++17
4  auto& [j, m, a] = p;           // par reference : ecrire j modifie p
5
6  std::map<std::string, date> agenda;
7  for (auto const& [nom, quand] : agenda) { /* ... */ }
```

Limitations

- Impossible de garantir qu'un objet est dans un état valide
- Impossible d'imposer une initialisation correcte
- Impossible de faire évoluer la représentation sans casser les clients

```
1  date d;  
2  d.jour  = 45;      // ce jour n'existe pas  
3  d.mois  = 13;     // ce mois n'existe pas  
4  d.annee = -7;     // discutable
```

LA QUESTION CENTRALE

Comment garantir qu'une date est toujours une date valide ?

Définition

- Une propriété que **toute fonction membre publique** trouve et laisse vraie

Mécanisme

- Les constructeurs l'établissent
- Une fonction membre peut le rompre le temps de son exécution, jamais au-delà

Exemples

- Une date représente un jour existant du calendrier
- Un `vector` a `size() <= capacity()` et un pointeur valide

Bénéfices

- Le code client peut raisonner sur le type sans lire son implémentation
- Sans invariant à protéger, un `struct` aux membres publics suffit

De struct vers class

Syntaxe

- struct : membres public par défaut, bases public par défaut
- class : membres private par défaut, bases private par défaut
- Aucune autre différence formelle entre les deux mots-clés

```
1  struct point { double x, y; };           // struct : x et y sont publics
2
3  // class : Les membres sont privées
4  class date
5  {
6      int jour_, mois_, annee_;
7      public: date(int j, int m, int a);    // seule porte d'entree
8  };
```

Usage

- struct pour un regroupement de données sans invariant
- class dès qu'un invariant est **à préserver**

Piliers de l'orienté objet

Encapsulation

- Cacher la représentation derrière une interface
- Sans elle, **aucun invariant** ne peut être garanti

Polymorphisme

- Traiter uniformément des types différents
- Le code appelant ignore **les types concrets**

Héritage

- Exprimer une relation de spécialisation
- **Un moyen d'obtenir le polymorphisme**, pas le seul

Modificateurs d'accès

public

- Accessible depuis n'importe quel code
- **L'interface du type** : ce sur quoi la classe s'engage

protected

- Accessible depuis la classe et ses classes dérivées
- **L'interface des sous-classes**, offerte à qui dérive

private

- Accessible uniquement depuis la classe elle-même
- **L'implémentation**, que la classe se réserve de changer

Que rendre public ou privé ?

Le critère

- Un membre dont **un invariant dépend** est privé
- Tout le reste peut être public

En pratique

- Les données portent l'état, donc presque toujours privées
- Les fonctions qui rendent le service sont publiques
- Un détail de représentation susceptible d'évoluer reste privé

L'EXCEPTION

L'agrégat sans invariant : tous ses membres sont publics, et c'est correct.

Une classe sans interface

Conséquences

- Les données sont `private` par défaut, hors d'atteinte du client
- Aucun code extérieur ne peut plus rompre l'invariant
- Sans fonction publique, **le type ne rend aucun service**

```
1  class date
2  {
3      int jour_, mois_, annee_;    // private par défaut
4  };
5
6  int main()
7  {
8      date d;                      // compile : Le constructeur engendre est public
9      d.jour_ = 25;                // ERREUR : membre prive
10 }
```

Principe

- La lecture est possible, la modification arbitraire ne l'est pas
- `const` garantit que la lecture ne modifie pas l'objet

```
1  class date
2  {
3      public:
4          int jour() const { return jour_; }
5          int mois() const { return mois_; }
6          int annee() const { return annee_; }
7
8      private:
9          int jour_ = 1, mois_ = 1, annee_ = 1970;
10 };
```

À PROSCRIRE

Un `set_mois(int)` redonnerait au client le pouvoir de poser un mois inexistant, donc de casser l'invariant.

Piège des accesseurs systématiques

Diagnostic

- Cette classe offre au client **les mêmes libertés qu'un struct public**
- Elle lui retire en plus l'initialisation par liste
- Aucun invariant n'est protégé : il ne reste que du bruit

```
1  class point
2  {
3      public:
4      double get_x() const { return x_; }
5      void set_x(double v) { x_ = v; }
6      // idem pour y
7      private:
8      double x_, y_;
9  };
10
11  point p{1.0, 2.0}; // ERREUR : ce n'est plus un agregat
```

LA QUESTION PRÉALABLE

Quel invariant la classe protège-t-elle ? Si la réponse est « aucun », un struct suffit.

Encapsulation comme fournisseur de service

Sémantique

- Les opérations nomment **le service rendu**, jamais la représentation
- L'invariant « le solde ne devient jamais négatif » est hors d'atteinte
- Un retrait refusé laisse le compte inchangé

```
1  class compte_bancaire
2  {
3      public:
4          long solde() const { return solde_centimes_; }
5
6          void deposer(long centimes);
7          bool retirer(long centimes);      // false si insuffisant, compte inchange
8          bool virer_vers(compte_bancaire& d, long c);  // maintient deux invariants
9
10     private:
11         long solde_centimes_ = 0;
12     };
```

Ruptures d'encapsulation

Fuites classiques

- Renvoyer une référence non-const ou un pointeur vers un membre privé
- Exposer un type interne dans la signature publique

```
1  class collection
2  {
3      public:
4          std::vector<int>& donnees() { return donnees_; }    // FUITE
5      private:
6          std::vector<int> donnees_;
7          long             somme_ = 0;    // invariant : somme des elements
8  };
9  collection c;
10 c.donnees().push_back(3);    // somme_ vaut toujours 0
```

Conséquences

- private ne suffit pas : **l'interface publique** décide seule
- Renvoyer un const& protège l'invariant mais fige la représentation

Fonctions membres constantes

Principe

- `const` garantit que la fonction ne modifie pas l'objet
- Seules les fonctions `const` sont **utilisables sur un objet `const`**

```
1  class compte_bancaire
2  {
3      public:
4          long solde() const { return solde_centimes_; }
5          std::string const& titulaire() const { return titulaire_; }
6          void deposer(long centimes);           // ne peut pas etre const
7      private:
8          std::string titulaire_;
9          long          solde_centimes_ = 0;
10 };
11
12 void imprimer(compte_bancaire const& c)
13 {
14     std::cout << c.titulaire() << " : " << c.solde(); // OK
15     c.deposer(100);                                // ERREUR : c est const
16 }
```

Refuser un état invalide

Le besoin

- Un constructeur n'a **pas de valeur de retour** où logger une erreur
- Une valeur d'erreur n'oblige pas l'appelant à la regarder
- `throw` signale l'échec sans que l'appelant puisse l'ignorer

```
1  date::date(int j, int m, int a) : jour_{j}, mois_{m}, annee_{a}
2  {
3      if (m < 1 || m > 12)
4          throw std::invalid_argument{"mois invalide"};
5  }
```

Exceptions standards

- `<stdexcept>` fournit les types usuels, tous dérivés de `std::exception`
- `invalid_argument`, `out_of_range` : une faute de l'appelant
- `runtime_error` : ce qui n'était détectable qu'à l'exécution
- `what()` renvoie le message et ne lève jamais

Mécanisme de l'exception

Déroulement

- throw interrompt le flot et **remonte la pile** jusqu'à un catch
- Chaque objet automatique traversé est détruit au passage
- Sans catch dans toute la pile, `std::terminate` est appelé
- Les destructeurs ne sont alors pas garantis

```
1  try
2  {
3      date d{25, 13, 2024};
4  }
5  catch (std::invalid_argument const& e)
6  {
7      std::cout << e.what() << "\n";
8  }
```

mois invalide

Mise en pratique : un créneau horaire

Invariant

- Un créneau est un intervalle de minutes dans la journée
- L'invariant tient en une ligne : $0 \leq \text{debut} < \text{fin} \leq 24 * 60$

```
1  class creneau
2  {
3      public:
4      creneau(int debut, int fin) : debut_{debut}, fin_{fin}
5      {
6          if (debut < 0 || fin > 24 * 60 || debut >= fin)
7              throw std::invalid_argument{"creneau invalide"};
8      }
9
10     int duree() const { return fin_ - debut_; }
11     bool chevauche(creneau const& a) const { return debut_ < a.fin_ && a.debut_ < fin_; }
12
13     private:
14     int debut_;    // minutes depuis minuit
15     int fin_;
16 };
```

Conception

- Le constructeur est le seul point d'entrée : il valide ou il refuse
- Établi à la construction, l'invariant n'a plus à être vérifié
- `duree` et `chevauche` sont des opérations `const`, pas des accesseurs

Subtilités

- Aucun accesseur ne renvoie `debut_` ni `fin_` : **rien n'y oblige**
- Un `set_fin` rouvrirait la porte que le constructeur vient de fermer

```
1  creneau matin{9 * 60, 12 * 60}, reunion{11 * 60 + 30, 13 * 60};
2  std::cout << std::boolalpha << matin.duree() << " " << matin.chevauche(reunion) << "\n";
3
4  try                                { creneau c{14 * 60, 9 * 60}; }
5  catch (std::exception const& e) { std::cout << e.what() << "\n"; }
```

```
180 true
creneau invalide
```

2. Constructeurs et destructeurs

Nécessité des constructeurs

Problématique

- Un objet doit être valide **dès sa création**
- L'initialisation par liste ne vérifie aucune contrainte

Constructeurs

- Fonction membre spéciale appelée à la création de l'objet
- Elle établit l'invariant, ou lève une exception

```
1  date d1{25, 12, 2024}; // l'invariant est établi
2  date d2{25, 13, 2024}; // leve : d2 n'existe jamais
```

PROPRIÉTÉ FONDAMENTALE

Soit le constructeur réussit, soit l'objet n'a jamais existé. Les membres déjà construits sont détruits au passage.

Qu'est-ce qu'un constructeur ?

Syntaxe

- Porte le **nom de la classe**, sans type de retour, pas même void
- Sans valeur de retour, l'échec ne peut se signaler que par une exception
- Peut être surchargé comme n'importe quelle fonction

```
1  class date
2  {
3      public:
4          date();           // 1er janvier 1970
5          date(int j, int m, int a); // jour, mois, annee
6          date(std::string_view iso); // "2024-12-25"
7
8      private:
9          int jour_ = 1, mois_ = 1, annee_ = 1970; // defaults deja valides
10 };
```

Liste d'initialisation des membres

Syntaxe

- Les membres sont **construits avant l'entrée dans le corps**
- L'ordre d'initialisation suit la déclaration, pas la liste
- Pour un membre `const` ou une référence, la liste est obligatoire
- Affecter dans le corps construit puis écrase, si le membre est une classe

```
1  date::date(int j, int m, int a) : jour_{j}, mois_{m}, annee_{a}    // liste
2  {
3      if (m < 1 || m > 12) throw std::invalid_argument{"mois invalide"};
4  }
5
6  // a éviter : les membres sont construits, puis écrases
7  // date::date(int j, int m, int a) { jour_ = j; mois_ = m; annee_ = a; }
```

Ordre d'initialisation

Principe

- L'ordre d'initialisation est celui **des déclarations**, pas celui de la liste

```
1  class tampon
2  {
3      public:
4      tampon(int n) : taille_{n}, data_(taille_) {}    // data_ recoit n'importe quoi
5
6      private:
7      std::vector<int> data_;    // declare en premier, donc construit en premier
8      int               taille_; // ... a partir d'un taille_ indetermine
9  };
10
11  // correction : declarer taille_ avant data_
```

PORTÉE DE -WREORDER

Seul -Wuninitialized reste quand la liste est réécrite dans l'ordre des déclarations : -Wreorder ne voit que l'ordre d'écriture.

Accolades ou parenthèses

Priorité

- Sous accolades, un constructeur `initializer_list` **l'emporte sur tous les autres**
- `vector` en possède un : `v{3, 0}` ne fait pas ce qu'il semble faire

```
1  std::vector<int> a(3, 0);    // trois elements valant 0
2  std::vector<int> b{3, 0};    // DEUX elements : 3 puis 0
```

Différences

- Les accolades interdisent **les conversions avec perte** : `int n{3.5};` est refusé
- `int n(3.5);` compile et donne 3, muet sans `-Wconversion`
- `T x();` déclare une fonction, `T x{};` déclare un objet

LA RÈGLE USUELLE

Accolades par défaut, parenthèses pour viser un constructeur précis, à commencer par ceux des conteneurs.

Initialiseurs de membres en classe

Règle

- Un initialiseur en classe donne **la valeur par défaut du membre**
- La liste d'initialisation du constructeur la remplace
- Un membre ajouté plus tard ne peut plus être oublié dans un constructeur
- Depuis C++14, un tel initialiseur n'empêche plus un agrégat

```
1  class compte_bancaire
2  {
3      public:
4      compte_bancaire(std::string t) : titulaire_{std::move(t)} {}
5      compte_bancaire(std::string t, long c)
6          : titulaire_{std::move(t)}, solde_centimes_{c} {}
7      private:
8      std::string titulaire_;
9      long        solde_centimes_ = 0;      // initialiseur en classe
10     bool        actif_          = true;
11 };
12 compte_bancaire a{"ana"};                // solde 0, actif true
13 compte_bancaire b{"bob", 25000};         // solde 25000, actif true
```

Constructeur par défaut

Définition

- Un constructeur par défaut s'appelle **sans argument**
- La classe en reçoit un automatiquement si elle n'en déclare aucun
- Déclarer un constructeur quelconque le fait disparaître

```
1  class date
2  {
3      public:
4          date() {}                // ne fait rien : les defaults suffisent
5          date(int j, int m, int a) : jour_{j}, mois_{m}, annee_{a} {}
6
7      private:
8          int jour_ = 1, mois_ = 1, annee_ = 1970;    // defaults valides
9  };
10
11  date d;    // 1/1/1970 : Les initialiseurs en classe s'appliquent
```

SANS LA LIGNE `DATE() {}`

Le constructeur à trois paramètres suffit à supprimer celui par défaut : `date d`; ne compile plus.

Validation dans le constructeur

Garanties

- Toute date construite désigne **un jour du calendrier**
- Aucune fonction membre n'a plus à le vérifier
- La garantie tombe dès qu'un mutateur touche les membres

```
1  class date
2  {
3      public:
4      date(int j, int m, int a) : jour_{j}, mois_{m}, annee_{a}
5      {
6          if (m < 1 || m > 12 || j < 1 || j > jours_du_mois(m, a))
7              throw std::invalid_argument{"date inexistante"};
8      }
9      private:
10     static int jours_du_mois(int m, int a);    // 28, 29, 30 ou 31
11     int jour_ = 1, mois_ = 1, annee_ = 1970;
12 };
13 date ok{29, 2, 2024};    // 2024 est bissextile
14 date ko{29, 2, 2023};    // leve : date inexistante
```

Conversions implicites

Problématique

- Un constructeur à **un seul argument** définit une conversion implicite
- `traiter(1024)` alloue 1024 octets sans que l'appel le laisse voir
- `traiter(true)` passe aussi, par la chaîne `bool` puis `int`

```
1  class bloc
2  {
3      public:
4      bloc(int taille) : octets_(taille) {}    // NON explicit
5      private:
6      std::vector<char> octets_;
7  };
8
9  void traiter(bloc const& b);
10
11  traiter(1024);    // compile : 1024 octets alloués, puis détruits
12  traiter(true);   // compile aussi : un bloc d'un seul octet
```

Correction par explicit

Effet

- `explicit` interdit la conversion implicite, pas la construction
- L'appelant doit **écrire le type qu'il construit**

```
1  class bloc
2  {
3      public:
4          explicit bloc(int taille) : octets_(taille) {}
5      private:
6          std::vector<char> octets_;
7  };
8
9  traiter(1024);           // ERREUR : plus de conversion implicite
10 traiter(true);           // ERREUR aussi
11 traiter(bloc{1024});     // intention ecrite noir sur blanc
```

LA RÈGLE

Tout constructeur à un seul argument est `explicit`, sauf si la conversion implicite est réellement voulue.

Délégation de constructeurs

Principe

- Un constructeur peut en appeler un autre de la même classe
- La validation n'existe alors qu'à **un seul endroit**
- Un constructeur qui délègue n'initialise aucun membre lui-même

```
1  class date
2  {
3      public:
4          date(int j, int m, int a);           // valide le calendrier
5
6          explicit date(int a) : date{1, 1, a} {} // 1er janvier de l'annee
7          date()                : date{1, 1, 1970} {}
8      private:
9          int jour_, mois_, annee_;
10 };
```

DÉLÉGATION CIRCULAIRE

Un cycle de délégations compile sans un mot, et se termine en débordement de pile.

Rôle

- Appelé automatiquement à la **fin de vie de l'objet**
- Libère les ressources acquises par l'objet
- Sortie de portée, delete, ou fin du programme pour un objet statique

Propriétés

- Un seul destructeur par classe, sans argument ni valeur de retour
- Il s'exécute aussi pendant la remontée d'une exception

```
1  class fichier
2  {
3      public:
4          explicit fichier(char const* nom);           // ouvre, ou leve
5
6          ~fichier() { if (f_) std::fclose(f_); }      // fclose(nullptr) est indefini
7      private:
8          std::FILE* f_ = nullptr;
9  };
```

Constructeurs et exceptions

Mécanisme

- Les membres **déjà construits** sont détruits, dans l'ordre inverse
- Le destructeur de la classe **n'est pas appelé** : l'objet n'a jamais existé
- Toute ressource appartient à un membre, jamais au corps du constructeur

```
1  class deux_fichiers
2  {
3      fichier a_;
4      fichier b_;    // si l'ouverture de b_ leve...
5      public:
6      deux_fichiers() : a_{"un.txt"}, b_{"absent.txt"} {}
7      ~deux_fichiers() { std::cout << "~deux_fichiers\n"; }
8  };
```

un.txt ouvert

un.txt ferme

exception : ouverture de absent.txt

Durée automatique

- Variables locales — détruites à la sortie du bloc
- Le cas normal, et celui qu'il faut privilégier

Durée dynamique

- Créée par `new`, détruite par `delete`
- À confier à un objet qui la libère dans son destructeur

Durée statique

- Variables globales et `static` — détruites à la fin du programme
- Attention à l'ordre d'initialisation entre unités de compilation

Durée liée au fil d'exécution

- `thread_local` — une instance par fil

Ordre de construction et de destruction

Un traceur

```
1  struct trace
2  {
3      char const* nom_;
4      trace(char const* n) : nom_{n} { std::cout << "+" << nom_ << "\n"; }
5      ~trace() { std::cout << "-" << nom_ << "\n"; }
6  };
7  int main()
8  {
9      trace a{"a"};
10     { trace b{"b"}; trace c{"c"}; }
11     trace d{"d"};
12 }
```

```
+a +b +c -c -b +d -d -a
```

Règles

- Destruction dans **l'ordre inverse de la construction**, à la sortie de portée
- Détruire à rebours garantit qu'une dépendance survit à qui s'en sert

Mise en pratique : une adresse IPv4

Trois façons de construire, un seul état

- Quatre octets, une valeur brute, ou rien du tout
- Les trois passent par la même validation

```
1  class adresse_v4
2  {
3      public:
4          adresse_v4() : adresse_v4(0, 0, 0, 0) {}           // delegation
5          adresse_v4(int a, int b, int c, int d)
6              : valeur_{octet(a) << 24 | octet(b) << 16 | octet(c) << 8 | octet(d)} {}
7
8          explicit adresse_v4(std::uint32_t brut) : valeur_{brut} {}
9      private:
10         static std::uint32_t octet(int v)
11         {
12             if (v < 0 || v > 255) throw std::out_of_range{"octet hors bornes"};
13             return static_cast<std::uint32_t>(v);
14         }
15
16         std::uint32_t valeur_ = 0;
17     };
```

Notions employées

- Le constructeur par **défaut délègue** : la validation n'existe qu'une fois
- `octet` est appelée dans la liste d'initialisation, avant tout corps
- `explicit` empêche qu'un entier devienne une adresse par mégarde
- `valeur_ = 0` en classe couvre tout constructeur qui l'oublierait

Subtilités

- La fonction de validation est `static` : elle sert avant que l'objet existe
- Une exception levée là interrompt la construction, sans objet à moitié fait

```
0.0.0.0  192.168.1.1  10.0.0.7
adresse_v4{192, 168, 1, 256} refuse : octet hors bornes
```

3. Héritage et polymorphisme dynamique

Définition du polymorphisme

Le mot

- Du grec : « qui prend plusieurs formes »
- Écrire **un seul code** qui fonctionne sur **plusieurs types**

Les formes en C++

- **Ad hoc** : surcharge de fonctions et d'opérateurs
- **Paramétrique** : templates — résolution à la compilation
- **Par sous-typage** : héritage et fonctions virtuelles — à l'exécution

Ce chapitre

- Le polymorphisme **par sous-typage**, dit dynamique
- C'est celui que l'on associe habituellement à l'orienté objet

Relation d'héritage

Syntaxe

```
1  // classe de base
2  class forme
3  {
4      public:
5          virtual ~forme() = default;
6          virtual double aire() const = 0;
7  };
8  class cercle : public forme    // classe derivee
9  {
10     public:
11         explicit cercle(double r) : r_{r} {}
12         double aire() const override { return 3.14159265 * r_ * r_; }
13     private:
14         double r_;
15     };
```

Sémantique

- cercle **est une** forme : substituable partout où forme est attendu

Fonctions virtuelles

Sans virtual : liaison statique

- La fonction appelée est déterminée par le **type déclaré**

Avec virtual : liaison dynamique

- La fonction appelée est déterminée par le **type réel de l'objet**
- La résolution a lieu à l'exécution

```
1  cercle c{2.0};  
2  forme const& f = c;      // une reference sur un objet bien vivant  
3  double a = f.aire();     // appelle cercle::aire grace a virtual
```

Condition indispensable

- L'appel doit passer **par une référence ou par un pointeur**
- Un objet manipulé par valeur perd son type dynamique

Polymorphisme en action

Un traitement uniforme d'objets distincts

```
1  std::mt19937 alea{2024};
2  std::uniform_real_distribution<double> cote{1.0, 5.0};
3  std::bernoulli_distribution pile{0.5};
4
5  std::vector<std::unique_ptr<forme>> formes;
6  for (int i = 0; i < 100; ++i)
7      if (pile(alea)) formes.push_back(std::make_unique<cercle>(cote(alea)));
8      else             formes.push_back(std::make_unique<rectangle>(cote(alea),
9                                                                    cote(alea)));
10 double total = 0.0;
11 for (auto const& f : formes) total += f->aire();  // type connu ici seulement
```

Sémantique

- La boucle ne **connaît aucun** type concret
- Ajouter un nouveau type de forme ne change pas cette boucle

Spécificateur override

override

- Indique explicitement que l'on redéfinit une fonction virtuelle de base
- Le compilateur **vérifie que c'est bien le cas**

```
1  class base { public: virtual void f(int) const; };
2
3  class sans_override : public base
4  { public: void f(int); };           // cache la virtuelle : -Woverloaded-virtual
5
6  class avec_override : public base
7  { public: void f(int) override; }; // ERREUR : Le const manque
```

LA RÈGLE

Toujours écrire override, sans exception.

Mot-clé final

Portée

- Sur une fonction : interdit toute redéfinition ultérieure
- Sur une classe : interdit d'en dériver

```
1  class cercle final : public forme      // plus aucune derivee possible
2  {
3      public:
4      explicit cercle(double r) : r_{r} {}
5      double aire() const override { return 3.14159265 * r_ * r_; }
6
7      private:
8      double r_;
9  };
10
11 void surface(cercle const& c) // cercle est final : aucune derivee
12 { std::cout << c.aire(); }   // possible, l'appel est devirtualise
```

LA RÈGLE

N'employer `final` que pour exprimer une intention de conception, jamais pour la dévirtualisation qu'il autorise.

Une fonction virtuelle pure

- `virtual double aire() const = 0;` déclare la fonction sans la définir
- La classe **devient abstraite** : impossible de l'instancier
- Toute classe dérivée **concrète doit** la définir

Une classe virtuelle pure

- Une classe dont toutes les fonctions sont virtuelles pures
- Plus un destructeur virtuel, et aucune donnée membre

```
1  class serialisable
2  {
3      public:
4          virtual ~serialisable() = default;
5          virtual std::string vers_json() const = 0;
6          virtual void depuis_json(std::string_view) = 0;
7  };
```

Destructeur virtuel

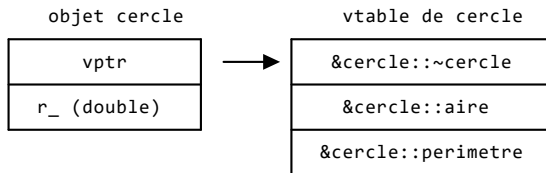
Problématique

- Une **base polymorphe** doit avoir un destructeur **virtual**
- Sinon un destructeur **protected non virtuel**, qui interdit cette destruction

```
1  class base    { public: ~base() {} };           // NON virtuel
2  class derive : public base
3  {
4      public:
5      ~derive() { /* Libere des ressources */ }
6      std::vector<int> data_;
7  };
8
9  base* p = new derive{};
10 delete p;           // COMPORTEMENT INDEFINI : ~derive n'est pas appele
```

Coût du polymorphisme dynamique

La table de fonctions virtuelles



Lecture

- Chaque objet polymorphe porte un **pointeur vers la table de son type**
- La table est unique par classe, partagée par toutes ses instances
- Un appel lit le vptr, puis l'entrée voulue, puis saute à l'adresse

Coût d'un appel virtuel

Impacts sur la performance

- **Espace** : un pointeur de plus par objet — 8 octets
- **Temps** : deux indirections mémoire par appel
- **Optimisation** : l'appel n'est en général pas *inlinable*
- **Prédiction** : le saut indirect peut être mal prédit

Ordre de grandeur

- Appel direct *inliné* : 0 cycle
- Appel virtuel bien prédit : 2 à 5 cycles
- Appel virtuel mal prédit : 15 à 20 cycles

Troncature d'un objet polymorphe

Le problème

```
1 // base CONCRETE : La troncature a lieu
2 class etiquette
3 { public: virtual ~etiquette() = default;
4     virtual std::string texte() const { return "etiquette"; } };
5 class datee : public etiquette
6 { public: std::string texte() const override { return "du 3 mars"; } };
7 void afficher(etiquette e) { std::cout << e.texte(); }
8 datee d; afficher(d);           // affiche "etiquette" : d est tronque
```

Mécanisme

- Seule la partie `etiquette` de `d` est copiée
- Le type **dynamique est perdu**, les appels virtuels retombent sur la base
- Avec une base abstraite, le code ne compile heureusement pas

LA RÈGLE

On manipule un objet polymorphe par référence ou par pointeur, jamais par valeur.

Principe de substitution de Liskov

Énoncé (Barbara Liskov, 1987)

- Si S est un sous-type de T, tout T peut être remplacé par un S
- Sans altérer **aucune propriété du programme**

En pratique

- Une classe dérivée ne doit pas affaiblir les garanties de la base
- Elle ne doit pas renforcer les pré-conditions
- Elle ne doit pas affaiblir les post-conditions
- Elle doit préserver les invariants de la base

LA RÈGLE

Le code écrit contre l'interface de base doit continuer de fonctionner sans connaître le type dérivé.

Liskov par l'exemple : le cas classique

Le rectangle et le carré

- Un carré est un rectangle à côtés égaux : la relation « est-un » semble acquise
- `carre` redéfinit les deux mutateurs pour maintenir cette égalité

```
1  class rectangle
2  {
3      public:
4          virtual ~rectangle() = default;
5          virtual void largeur(double l) { l_ = l; }
6          virtual void hauteur(double h) { h_ = h; }
7          double aire() const { return l_ * h_; }
8      protected:
9          double l_ = 0.0, h_ = 0.0;
10 };
11
12 class carre : public rectangle    // "un carre EST un rectangle"
13 {
14     public:
15         void largeur(double l) override { l_ = h_ = l; }
16         void hauteur(double h) override { l_ = h_ = h; }
17 };
```

Liskov par l'exemple : la mise en défaut

Source du problème

```
1 void tester(rectangle& r)
2 {
3     r.largeur(5.0);
4     r.hauteur(4.0);
5     assert(r.aire() == 20.0);    // vrai pour tout rectangle
6 }
7
8 int main()
9 {
10     rectangle r; tester(r);    // OK
11     carre      c; tester(c);    // ECHEC : aire() vaut 16
12 }
```

Diagnostic

- carre **affaiblit une post-condition** de rectangle
- Après largeur(1), la hauteur d'un rectangle doit rester inchangée
- L'héritage est incorrect ici : « est-un » en maths n'est pas « est-un » en C++

Liskov : les corrections possibles

Option 1 — supprimer les mutateurs

- **Rendre rectangle immuable** : plus de post-condition à violer
- Un carré immuable est bien un rectangle immuable

Option 2 — supprimer la relation d'héritage

- carre et rectangle sont deux types indépendants
- Une conversion explicite `carre::vers_rectangle()` si nécessaire

Option 3 — inverser la hiérarchie

- Une base abstraite `forme`, dont `carre` et `rectangle` dérivent séparément

LA RÈGLE

La hiérarchie est dictée par le **comportement**, pas par la taxonomie.

Le doute doit conduire à la composition plutôt qu'à l'héritage.

Héritage ou composition ?

Les deux formes d'héritage

- **public** — « est-un » : la seule qui exprime le sous-typage
- **private** — « est implémenté en termes de » : pas de substituabilité
- La composition le remplace toujours

Le test décisif

- Le client doit-il utiliser le nouveau type **à travers** l'interface de l'ancien ?
 - Oui → héritage public
 - Non → composition

Le coût de l'héritage

- C'est le **couplage le plus fort du langage**, figé à la compilation
- La composition laisse changer le composant à l'exécution

Gestion des objets polymorphes

Le contexte

- Un objet polymorphe est manipulé par pointeur ou référence
- Sa durée de vie doit être gérée explicitement

La mauvaise solution

```
1  std::vector<forme*> formes;  
2  formes.push_back(new cercle{1.0});  
3  // ... traitements pouvant lever une exception ...  
4  for (auto p : formes) delete p;      // jamais atteint en cas d'exception
```

La bonne solution

- `std::unique_ptr<forme>` en cas de propriétaire unique
- `std::shared_ptr<forme>` en cas de propriété partagée
- Une référence ou un pointeur nu **non propriétaire** pour l'observation

Sémantique

- **Propriété exclusive** : un seul propriétaire à la fois
- Non copiable, **seulement déplaçable**
- **Aucun surcoût** par rapport à un pointeur nu, en taille comme en vitesse

Exemple

```
1  std::unique_ptr<forme> creer(std::string_view type)
2  {
3      if (type == "cercle")    return std::make_unique<cercle>(1.0);
4      if (type == "rectangle") return std::make_unique<rectangle>(2.0, 3.0);
5      return nullptr;
6  }
7
8  auto f = creer("cercle");
9  if (f) std::cout << f->aire() << "\n";  // détruit en fin de portée
```

Sémantique

- **Propriété partagée** avec comptage de références
- L'objet est détruit quand le dernier propriétaire disparaît
- **Deux pointeurs** au lieu d'un, et un compteur atomique par copie

Exemple

```
1  std::shared_ptr<cercle> a = std::make_shared<cercle>(1.0); // compteur = 1
2
3  std::vector<std::shared_ptr<forme>> registre;
4  registre.push_back(a); // compteur = 2
5  {
6      auto b = a; // compteur = 3
7      std::cout << b.use_count() << "\n"; // affiche 3
8  } // compteur = 2
9
10 registre.clear(); // compteur = 1
11 a.reset(); // compteur = 0 : destruction
```

Mise en pratique : une paie polymorphe

Le besoin

- Trois contrats, trois formules ; le service de paie n'en connaît aucune

```
1  class employe
2  {
3      public:
4          explicit employe(std::string nom) : nom_{std::move(nom)} {}
5          virtual ~employe() = default;
6          virtual double paie() const = 0;          // chaque contrat a sa formule
7          std::string const& nom() const { return nom_; }
8          private:
9              std::string nom_;
10 };
11
12 // taux x heures, majoré de 50 % au-delà de 35 h
13 class employe_horaire    final : public employe { /* ... */ };
14 // salaire mensuel fixe
15 class employe_mensuel    final : public employe { /* ... */ };
16 // fixe + pourcentage du chiffre d'affaires
17 class employe_commission final : public employe { /* ... */ };
```

Mise en pratique : le traitement

La boucle de paie

- Elle appelle paie() sans savoir de quel contrat il s'agit

```
1  std::vector<std::unique_ptr<employe>> paies;
2  paies.push_back(std::make_unique<employe_mensuel>("ana", 2800.0));
3  paies.push_back(std::make_unique<employe_horaire>("bob", 18.0, 40));
4  paies.push_back(std::make_unique<employe_commission>("cleo", 1200.0,
5                                                         26000.0, 0.06));
6  double total = 0.0;
7  for (auto const& e : paies)
8  {
9      std::cout << e->nom() << " " << e->paie() << "\n";
10     total += e->paie();
11 }
12 std::cout << "total " << total << "\n";
```

ana	2800.00
bob	765.00
cleo	2760.00
total	6325.00

Notions employées

- paie est **virtuelle pure** : la base ne sait pas calculer, elle l'exige
- Le destructeur est virtuel, sans quoi la destruction par la base fuirait
- `final` sur les dérivées interdit une redéfinition de plus
- La boucle de synthèse ne **connaît aucun des trois types concrets**

Subtilités

- Les objets sont tenus par `unique_ptr` : les copier découperait l'objet
- Ajouter un mode de rémunération n'ouvre aucun fichier existant

4. Sémantique de valeur, copie et déplacement

C++ est un langage de valeurs

Le modèle par défaut

- Une **variable contient** une valeur, elle ne la référence pas
- **L'affectation copie**, elle ne partage pas

```
1  std::vector<int> a{1, 2, 3};  
2  std::vector<int> b = a;      // b est une COPIE independante  
3  b.push_back(4);            // a est inchange
```

Comparaison

- En Java ou Python, `b = a` **crée un alias** sur le même objet
- En C++, c'est une copie complète

Les bénéfices

- Raisonnement local : aucune modification à distance
- Aucune indirection, donc localité mémoire optimale

L'inquiétude classique

```
1  std::vector<double> calculer(std::size_t n)
2  {
3      std::vector<double> resultat(n);
4      // ... remplissage ...
5      return resultat;           // une copie de n doubles ?
6  }
7
8  auto v = calculer(1'000'000); // une copie de plus ?
```

La réponse

- **Aucune copie** n'a lieu
- Deux mécanismes s'en chargent : l'élimination de copie, puis le déplacement

RVO — Return Value Optimization

- Le compilateur construit **l'objet directement** dans l'emplacement du destinataire
- Depuis C++17, **l'élision est obligatoire** pour les temporaires

```
1  std::vector<double> f() { return std::vector<double>(1000); }  
2  auto v = f();           // garanti : construction directe dans v, zero copie
```

NRVO — Named Return Value Optimization

- Même mécanisme pour une variable **locale nommée**
- **Autorisée mais non obligatoire** : les compilateurs usuels l'appliquent

```
1  std::vector<double> g()  
2  {  
3      std::vector<double> r(1000);  
4      return r;           // NRVO : construction directe dans le destinataire  
5  }
```

Obstacles à la NRVO

Deux candidats au retour

- Le compilateur ne sait pas dans lequel des deux construire le résultat
- Il retombe sur une copie

```
1  std::vector<double> h(bool cond)
2  {
3      std::vector<double> a(1000), b(1000);
4      return cond ? a : b;      // deux candidats : pas de NRVO
5  }
```

Les cas exclus par la norme

- Un paramètre de la fonction : son emplacement est fixé par l'appelant
- Tout sauf une **variable locale automatique** : un membre, un élément de conteneur

Élision garantie depuis C++17

Avant C++17 : une optimisation permise

- Le **compilateur pouvait** supprimer la copie, sans y être obligé
- Un type **ni copiable ni déplaçable** ne pouvait pas être renvoyé par valeur

Depuis C++17 : ce n'est plus une optimisation

- $T\ x = T\{\}$; ne construit **qu'un seul** objet, par définition du langage
- Aucun temporaire à élider : $T\{\}$ initialise directement la destination

```
1  struct inamovible { inamovible() = default; inamovible(inamovible const&) = delete; };
2
3  inamovible fabrique() { return inamovible{}; }    // legal depuis C++17 seulement
4  auto obj = fabrique();                          // un seul objet construit
```

Limitations

- **La NRVO**, sur une variable nommée, reste une optimisation permise

lvalue et rvalue

lvalue

- Une expression qui désigne un **objet identifiable** et persistant
- On peut prendre son adresse
- Exemples : une variable, *p, tab[i]

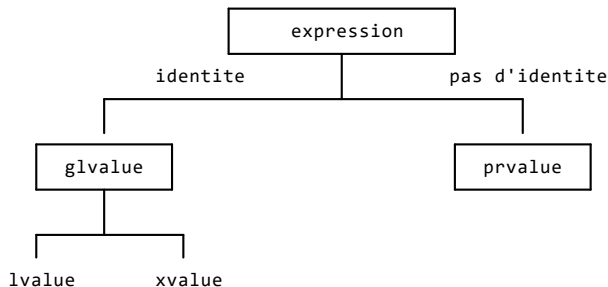
rvalue

- Une expression désignant une **valeur temporaire**
- Elle va disparaître à la fin de l'expression complète
- Exemples : 42, a + b, f() renvoyant par valeur

```
1  int  x = 10;      // x est une lvalue, 10 est une rvalue
2  int* p = &x;      // OK : on peut adresser une lvalue
3  // int* q = &10;   // ERREUR : on ne peut pas adresser une rvalue
```

Deux questions indépendantes

- L'expression **a-t-elle une identité** ? peut-on en prendre l'adresse ?
- Ses ressources **peuvent-elles être transférées** ?
- Les deux transférables, xvalue et prvalue, forment les rvalue



Reconnaître une catégorie

Expression	Catégorie	Pourquoi
<code>x</code>	<i>lvalue</i>	a un nom, donc une identité
<code>x + 1</code>	<i>prvalue</i>	résultat temporaire, sans nom
<code>f()</code> renvoyant <code>std::string&</code>	<i>lvalue</i>	désigne un objet existant
<code>std::move(x)</code>	<i>xvalue</i>	a une identité, ressources transférables
<code>t[i], *p</code>	<i>lvalue</i>	désigne un emplacement

Les regroupements utiles

- **glvalue** = ce qui a une identité : *lvalue* et *xvalue*
- **rvalue** = ce dont on peut transférer les ressources : *xvalue* et *prvalue*
- Une **référence rvalue** se lie aux *rvalues*, une **référence constante** à tout

Références rvalue

Notation

- `std::string&` — référence vers une lvalue
- `std::string&&` — référence vers une rvalue

```
1 void f(std::string& s) { std::cout << "lvalue\n"; }
2 void f(std::string&& s) { std::cout << "rvalue\n"; }
3
4 std::string s{"bonjour"};
5 f(s); // lvalue
6 f(std::string{"salut"}); // rvalue
7 f(std::move(s)); // rvalue
```

Bénéfices

- Le code peut **savoir que son argument est un temporaire**
- Il **peut donc en transférer** les ressources sans risque

La question posée à chaque appel

- Quelle surcharge le compilateur retient-il selon la catégorie de l'argument ?

Argument	<code>std::string&</code>	<code>std::string const&</code>	<code>std::string&&</code>
lvalue	oui	oui	non
lvalue constante	non	oui	non
prvalue ou xvalue	non	oui	oui

Copie ou déplacement

- `std::string const&` retenu : l'objet est **copié**, il survit à l'appel
- `std::string&&` retenu : l'objet est **déplacé**, l'appelé peut le vider
- Quand les deux conviennent, la **référence rvalue l'emporte** sur la constante

std::move n'est pas un déplacement

Sémantique

- Une simple **conversion de type** vers T&&
- Elle ne **génère aucune** instruction machine
- Elle ne déplace rien par elle-même

```
1  template<typename T>
2  constexpr std::remove_reference_t<T>&& move(T&& t) noexcept
3  {
4      return static_cast<std::remove_reference_t<T>&&>(t);
5  }
```

Contrat

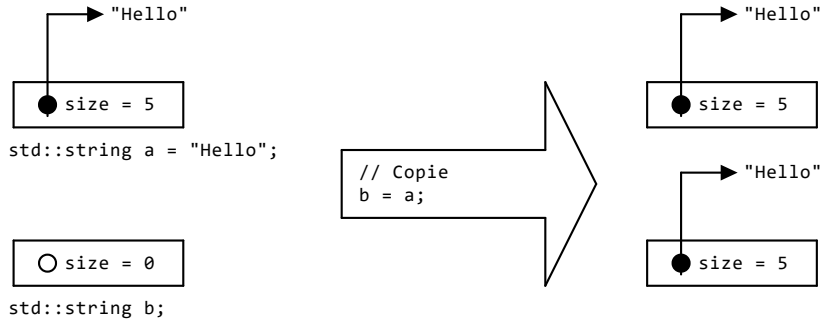
- « Je m'engage à ne plus utiliser la valeur de cet objet »
- Le déplacement effectif revient au constructeur ou à l'affectation sélectionnés

Problématiques

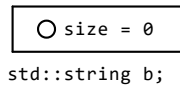
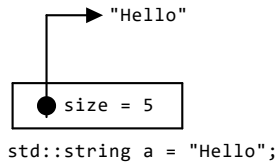
- Copier un objet dont la source ne resservira pas est du travail perdu
- Le déplacement transfère les ressources au lieu de les dupliquer
- Copier une chaîne d'un mégaoctet : **une allocation**, un million d'octets
- La déplacer : **trois affectations**, quelle que soit sa taille

```
1  std::vector<std::string> v;  
2  std::string s = construire_grande_chaine();  
3  v.push_back(s);           // COPIE : allocation + memcpy  
4  v.push_back(std::move(s)); // DEPLACEMENT : transfert de pointeur
```

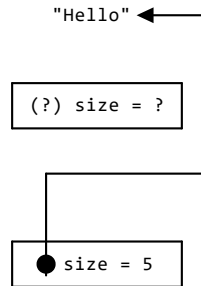
Affectation par copie



Affectation par transfert



// Transfert
`b = std::move(a);`



État après déplacement

Déroulement du déplacement

- Un objet déplacé est dans un état **valide mais non spécifié**
- On peut le détruire et lui affecter une nouvelle valeur
- On ne doit **rien supposer de son contenu**

```
1  std::string a{"bonjour"};
2  std::string b = std::move(a);
3
4  // a.size() est valide mais sa valeur n'est pas garantie
5  a = "nouvelle valeur";      // OK : reaffectation autorisee
6  std::cout << a;            // OK apres reaffectation
```

En pratique

- Les implémentations laissent généralement les conteneurs vides
- S'appuyer là-dessus reste une **erreur de portabilité**

std::move sur un const ne déplace rien

Mécanisme

- `std::move(source)` produit une `std::string const&&`
- Le constructeur de déplacement prend une `std::string&&` : il ne convient pas
- Le constructeur de copie prend une `std::string const&` : **c'est lui qui gagne**

```
1  std::string const source = "un texte bien trop long pour le tampon interne";  
2  std::string cible = std::move(source);  // COPIE, en silence
```

Problèmes

- Aucun avertissement, aucune erreur — juste une copie que l'on croyait évitée
- Un membre `const` d'une classe rend de même tout déplacement impossible

Quand ne pas écrire `std::move`

Deux retours, un seul correct

```
1  std::vector<int> mauvais() { std::vector<int> v(1000); return std::move(v); }
2  std::vector<int> bon()    { std::vector<int> v(1000); return v; }
```

Problématiques

- `return v;` **autorise la NRVO** : aucune construction supplémentaire
- `return std::move(v);` renvoie une *xvalue*, ce qui **interdit la NRVO**
- On force alors un déplacement là où il n'y avait rien à faire
- GCC et Clang le signalent avec `-Wpessimizing-move`

LA RÈGLE

Sur un retour de variable locale, on écrit le nom, rien d'autre. `std::move` au retour ne se justifie que pour un paramètre de la fonction, que la NRVO ne couvre jamais.

Déplacer un membre proprement

Le membre laissé dans un état valide

```
1  class tampon
2  {
3      std::vector<char> data_;
4      std::size_t      taille_;
5
6  public:
7      tampon(tampon&& autre) noexcept
8          : data_ {std::move(autre.data_)} // vole le tableau
9            , taille_{autre.taille_}      // copie l'entier : rien a voler
10     {
11         autre.taille_ = 0;                // remet la source en état cohérent
12     }
13 };
```

Les deux gestes

- `std::move` sur chaque **membre qui possède** une ressource
- Remettre à zéro les membres scalaires : le déplacement ne les touche pas
- Sans cela, la source reste incohérente

Auto-déplacement

Deux façons d'y arriver

```
1  v = std::move(v);           // rare, mais possible via un alias
2  std::swap(t[i], t[j]);      // devient une auto-affectation si i == j
```

Exigence de la norme

- Après auto-déplacement, l'objet **reste valide**, sa valeur n'est pas spécifiée
- Le programme ne doit pas planter, mais rien ne peut être supposé du contenu

La conséquence pratique

- Écrite naïvement, libérer puis voler détruit la ressource avant de la voler
- *copy and swap* et le test `if (this != &autre)` l'évitent tous deux

Quand copie-t-on encore ?

Situation	Ce qui se passe
<code>T b = a;</code> avec <code>a</code> nommé	copie
<code>T b = f();</code>	ni copie ni déplacement
<code>return v;</code> variable locale	NRVO, sinon déplacement
<code>return std::move(v);</code>	déplacement forcé
<code>std::move</code> sur un <code>const</code>	copie silencieuse
Membre <code>const</code> dans la classe	copie , déplacement impossible
Passage par <code>T const&</code>	aucune copie

La règle qui résume tout

- On ne « fait » pas un déplacement : on le **rend possible**
- Le compilateur le fait alors quand il le peut

Mise en pratique : une matrice dense

Un type à sémantique de valeur

- Les coefficients vivent dans un `std::vector`, membre du type
- Rien n'est écrit pour la copie, le déplacement ni la destruction

```
1  class matrice
2  {
3  public:
4      matrice(std::size_t lignes, std::size_t colonnes)
5          : lignes_{lignes}, colonnes_{colonnes}, valeurs_(lignes * colonnes, 0.0) {}
6      double& operator()(std::size_t i, std::size_t j) { return valeurs_[i * colonnes_ + j]; }
7      std::size_t lignes() const { return lignes_; }
8      std::size_t colonnes() const { return colonnes_; }
9  private:
10     std::size_t      lignes_, colonnes_;
11     std::vector<double> valeurs_;
12 };
13 matrice transposee(matrice const& m)
14 {
15     matrice t{m.colonnes(), m.lignes()}; // ... remplissage ...
16     return t;                          // NRVO : ni copie ni déplacement
17 }
```

Notions employées

- matrice $b = a$; produit **une copie complète** : modifier b laisse a intact
- transposee renvoie un objet local : la NRVO le construit chez l'appelant
- Écrire `return std::move(t)`; empêcherait cette élision au lieu de l'aider
- matrice $c = \text{std::move}(b)$; transfère le vecteur sans toucher aux coefficients

Subtilités

- Après déplacement, b reste **détruisible et affectable**, rien de plus
- Lire son contenu est légal mais n'a aucun sens : on ne le fait pas

```
a(0,0)=1 b(0,0)=9  
transposee 3x2, t(2,1)=5  
c 2x3, c(0,0)=9
```

5. Fonctions membres spéciales

Six fonctions membres spéciales

Le compilateur peut les générer

```
1  class T
2  {
3  public:
4      T();                // constructeur par défaut
5      ~T();              // destructeur
6      T(T const&);        // constructeur de copie
7      T& operator=(T const&); // affectation par copie
8      T(T&&) noexcept;    // constructeur de déplacement
9      T& operator=(T&&) noexcept; // affectation par déplacement
10 };
```

Le point délicat

- Les règles de génération implicite sont **complexes et peu intuitives**
- Déclarer l'une supprime souvent la génération d'une autre

Special Members

compiler implicitly declares

user declares		default constructor	destructor	copy constructor	copy assignment	move constructor	move assignment
	Nothing	defaulted	defaulted	defaulted	defaulted	defaulted	defaulted
	Any constructor	not declared	defaulted	defaulted	defaulted	defaulted	defaulted
	default constructor	user declared	defaulted	defaulted	defaulted	defaulted	defaulted
	destructor	defaulted	user declared	defaulted	defaulted	not declared	not declared
	copy constructor	not declared	defaulted	user declared	defaulted	not declared	not declared
	copy assignment	defaulted	defaulted	defaulted	user declared	not declared	not declared
	move constructor	not declared	defaulted	deleted	deleted	user declared	not declared
	move assignment	defaulted	defaulted	deleted	deleted	not declared	user declared

Constructeur de copie

Une copie profonde

```
1  class tampon
2  {
3  public:
4      explicit tampon(std::size_t n) : n_{n}, data_{new double[n]{} } {}
5      tampon(tampon const& autre) : n_{autre.n_}, data_{new double[autre.n_]}
6      {
7          std::copy(autre.data_, autre.data_ + n_, data_);
8      }
9      ~tampon() { delete[] data_; }
10
11     private:
12         std::size_t n_;
13         double*     data_;
14 };
```

Subtilités

- **Copie profonde** : sans cela, double libération à la destruction
- `n_` est initialisé avant `data_` : respecter l'ordre de déclaration

Affectation par copie

L'opérateur d'affectation

```
1 tampon& operator=(tampon const& autre)
2 {
3     if (this == &autre) return *this;    // auto-affectation
4
5     double* nouveau = new double[autre.n_]; // allouer AVANT de detruire
6     std::copy(autre.data_, autre.data_ + autre.n_, nouveau);
7
8     delete[] data_;
9     data_ = nouveau;
10    n_ = autre.n_;
11    return *this;
12 }
```

Les trois exigences

- Gérer l'**auto-affectation**
- Rester correct si **l'allocation échoue**
- Renvoyer `*this` par référence

Idiome copy and swap

Une seule fonction pour les deux

```
1  // membres de la classe tampon
2  friend void swap(tampon& a, tampon& b) noexcept
3  {
4      using std::swap;
5      swap(a.n_,    b.n_);
6      swap(a.data_, b.data_);
7  }
8
9  tampon& operator=(tampon autre) noexcept // par VALEUR : la copie
10 {                                         // ou le déplacement est fait
11     swap(*this, autre);                  // par l'appelant
12     return *this;
13 }
```

Bénéfices

- Une seule fonction gère **copie et déplacement**
- Auto-affectation correcte par construction
- **Garantie forte** face aux exceptions

Constructeur de déplacement

Le transfert des ressources

```
1 tampon(tampon&& autre) noexcept
2     : n_{autre.n_}, data_{autre.data_}
3 {
4     autre.n_ = 0;
5     autre.data_ = nullptr;    // ESSENTIEL : la source ne possede plus rien
6 }
```

Pourquoi noexcept est indispensable

- `std::vector` ne déplace à la réallocation que si le déplacement est `noexcept`
- Sinon il **retombe sur la copie** pour garantir la sûreté
- Un constructeur de déplacement sans `noexcept` est souvent inutilisé

Affectation par déplacement

La libération avant transfert

```
1 tampon& operator=(tampon&& autre) noexcept
2 {
3     if (this == &autre) return *this;
4
5     delete[] data_;           // Libérer l'ancien contenu
6
7     n_ = autre.n_;           // voler les ressources
8     data_ = autre.data_;
9
10    autre.n_ = 0;             // neutraliser la source
11    autre.data_ = nullptr;
12
13    return *this;
14 }
```

La séquence à respecter

- Libérer, voler, neutraliser — dans cet ordre

Règle de trois

Énoncé (C++98)

- Qui a **besoin de l'un** des trois suivants a généralement **besoin des trois** :
 - destructeur
 - constructeur de copie
 - affectation par copie

La logique

- Avoir besoin d'un destructeur signifie gérer une ressource
- Gérer une ressource signifie que la copie par défaut est fautive

Règle de cinq

Énoncé (C++11)

- Aux trois précédents s'ajoutent les deux opérations de déplacement
- En écrire une, c'est les écrire toutes les cinq

En pratique

- Déclarer un **destructeur supprime** la génération du déplacement
- Déclarer le destructeur et la copie seuls supprime le déplacement
- La perte d'optimisation est silencieuse

```
1  class ressource
2  {
3      public:
4          ~ressource();
5          ressource(ressource const&);
6          ressource& operator=(ressource const&);
7          ressource(ressource&&) noexcept;
8          ressource& operator=(ressource&&) noexcept;
9  };
```

= default : effets réels

Deux déclarations voisines

```
1 struct A { A(A const&) = default; };           // declaree, generee par le compilateur
2 struct B { B(B const&) {} };                   // declaree ET definie a la main
```

La différence n'est pas cosmétique

- A reste **trivialement copiable** : la copie peut se faire par memcpy
- B ne l'est plus, même si le corps est vide

Conséquences

- Un type trivialement copiable peut être copié en bloc par les conteneurs
- `std::is_trivially_copyable_v<A>` vaut true, celui de B vaut false

La règle

- Si le comportement engendré convient, `= default` plutôt qu'un corps vide

= delete : supprimer plutôt qu'interdire

Un type non copiable

```
1  class verrou
2  {
3  public:
4      verrou(verrou const&)          = delete;    // non copiable
5      verrou& operator=(verrou const&) = delete;
6      verrou(verrou&&)              = default;    // mais déplacable
7      verrou& operator=(verrou&&)    = default;
8  };
```

Pourquoi pas un private sans définition

- Technique d'avant C++11 : l'erreur tombait à l'édition de liens, illisible
- = delete produit une erreur **de compilation**, à la ligne fautive

Un autre usage

- Supprimer une surcharge précise interdit une conversion implicite
- `void f(char) = delete;` bloque `f('a')` sans toucher à `f(int)`

noexcept sur le déplacement

La promesse écrite dans la signature

```
1  class tampon
2  {
3  public:
4      tampon(tampon&&) noexcept;           // sans le noexcept, tout change
5      tampon& operator=(tampon&&) noexcept;
6  };
```

Pourquoi cette promesse est tenable

- Un déplacement ne **fait que transférer des pointeurs déjà alloués**
- Il n'alloue rien, donc il ne peut pas échouer par manque de mémoire

Pourquoi elle est indispensable

- La bibliothèque utilise `std::move_if_noexcept`
- Sans la promesse, **elle copie** au lieu de déplacer
- La slide suivante en montre le coût mesurable

Croissance de vector, en détail

Mécanisme de réallocation

- vector alloue un nouveau tableau, y transfère les éléments, détruit l'ancien

Le dilemme

- **Si un déplacement lève** au milieu du transfert, les éléments sont perdus
- L'ancien tableau ne les contient plus
- vector ne pourrait donc plus offrir la garantie forte

```
deplacement noexcept -> vector DEPLACE : O(1) par element  
deplacement lancable -> vector COPIE   : copie chaque element
```

La conséquence

- Un noexcept oublié sur un type coûteux à copier coûte un ordre de grandeur
- Et aucun message ne le signale

Un type non copiable mais déplaçable

Une poignée de descripteur

```
1  class socket
2  {
3      int fd_ = -1;
4  public:
5      explicit socket(int fd) noexcept : fd_{fd} {}
6      ~socket() { if (fd_ != -1) ::close(fd_); }
7      socket(socket const&) = delete;
8      socket& operator=(socket const&) = delete;
9
10     socket(socket&& a) noexcept : fd_{std::exchange(a.fd_, -1)} {}
11     socket& operator=(socket&& a) noexcept
12     { std::swap(fd_, a.fd_); return *this; }
13 };
```

Pourquoi c'est le bon modèle

- Un descripteur ne se duplique pas : la copie n'a **pas de sens**
- `std::exchange` vole la valeur et remet la source en état neutre

Quelle fonction membre écrire, et quand

Situation	À écrire
Des membres qui se gèrent seuls	rien — règle de zéro
Une ressource brute possédée	les cinq, déplacement noexcept
La copie n'a pas de sens	copie = delete, déplacement écrit
Classe de base polymorphe	destructeur virtual, copie protégée

Le piège à connaître

- Une classe qui déclare son destructeur retombe **silencieusement en copie seule**

Règle de zéro

Énoncé

- La meilleure classe est celle qui ne **déclare aucune de ces six fonctions**
- Elle délègue la gestion des ressources à des membres qui savent le faire

```
1  class document
2  {
3      public:
4          document(std::string t, std::size_t n) : titre_{std::move(t)}, contenu_(n) {}
5          // aucune fonction speciale : tout est correct par default
6
7      private:
8          std::string          titre_;
9          std::vector<std::string> contenu_;
10 };
```

Bénéfices

- Copie, déplacement, destruction, tous corrects et optimaux
- Aucune ligne de code à maintenir ni à tester

Règle de zéro : conséquences

La stratégie de conception

- Écrire **une classe minimale par ressource brute**
- Elle l'acquiert dans son constructeur, la libère dans son destructeur
- Composer ces classes dans les types métier
- Les types métier suivent alors la règle de zéro

La séparation des responsabilités

- Une classe gère **soit une ressource, soit de la logique métier**
- Jamais les deux à la fois
- C'est une application directe du principe de responsabilité unique

Le résultat

- 95 % des classes d'un code moderne ne déclarent aucune fonction spéciale

Mise en pratique : zéro d'un côté, cinq de l'autre

Deux types, deux régimes

- profil n'agrège que des membres qui se gèrent eux-mêmes
- zone possède un tableau nu, donc les cinq fonctions lui incombent

```
1  // regle de zero : rien a ecrire
2  class profil
3  {
4      public:
5          profil(std::string nom, std::vector<std::string> etiquettes)
6              : nom_{std::move(nom)}, etiquettes_{std::move(etiquettes)} {}
7          std::string const& nom() const { return nom_; }
8          private:
9              std::string          nom_;
10             std::vector<std::string> etiquettes_;
11 };
12
13 profil q = p;                // copie engendree, correcte
14 profil r = std::move(p);     // déplacement engendre, correct
```

Autre régime : les cinq à écrire

Contraintes

- Un `double*` nu ne sait ni se copier, ni se déplacer, ni se libérer
- L'affectation par valeur plus `swap` couvre la copie et le déplacement

```
1  class zone
2  {
3  public:
4  explicit zone(std::size_t n) : n_{n}, donnees_{new double[n]} {}
5  zone(zone const& a) : n_{a.n_}, donnees_{new double[a.n_]}
6  { std::copy(a.donnees_, a.donnees_ + n_, donnees_); }
7  zone(zone&& a) noexcept
8      : n_{std::exchange(a.n_, 0)}, donnees_{std::exchange(a.donnees_, nullptr)} {}
9  zone& operator=(zone a) noexcept { swap(a); return *this; }
10 ~zone() { delete[] donnees_; }
11 double& operator[](std::size_t i) { return donnees_[i]; }
12 std::size_t taille() const { return n_; }
13 void swap(zone& a) noexcept { std::swap(n_, a.n_); std::swap(donnees_, a.donnees_); }
14 private:
15 std::size_t n_ = 0;
16 double* donnees_ = nullptr;
17 };
```

Notions employées

- profil obtient les six fonctions engendrées, toutes correctes
- zone déclare un destructeur : sans les autres, le déplacement disparaîtrait
- `std::exchange` laisse la source dans un état **valide et connu**
- `noexcept` sur le déplacement permet à `vector` de déplacer plutôt que copier

Subtilités

- Le seul critère est la **possession d'une ressource**, pas la taille du type
- `zone` existe pour l'exercice : en pratique un `vector` ferait le travail

```
1  profil p{"ana", {"c++", "hpc"}};  
2  profil q = p;           // copie engendree  
3  zone  a{4};  a[0] = 3.5;  
4  zone  b = std::move(a); // déplacement écrit a la main  
5  std::cout << q.nom() << " " << b[0] << " " << b.taille()  
6          << " " << a.taille() << "\n";
```

6. RAII et sûreté face aux exceptions

Resource Acquisition Is Initialization

- L'acquisition d'une ressource se **fait dans un constructeur**
- Sa libération se **fait dans le destructeur**
- La durée de vie de la ressource est celle de l'objet

Garanties

- Aucune fuite quel que soit le chemin de sortie
- Retour normal, return anticipé, break, exception qui traverse
- Aucune libération oubliée ni double libération

Mécanisme réel

- Le mécanisme essentiel est en réalité la **destruction déterministe**
- Certains proposent le nom *Scope-Bound Resource Management*

Bien plus que la mémoire

- Mémoire dynamique — `unique_ptr`, `vector`, `string`
- Descripteurs de fichiers — `fstream`
- Verrous — `lock_guard`, `unique_lock`, `scoped_lock`
- Connexions réseau, transactions de base de données
- Contextes graphiques, handles système
- Toute paire « acquérir / relâcher »

Le critère

- Dès qu'une **opération doit être annulée** en fin d'usage, il faut un objet RAI

RAII : un exemple complet

Un chronomètre

- Le constructeur note l'instant de départ, le destructeur affiche la durée
- Aucune fonction à appeler : la portée décide de la mesure

```
1  class chronometre
2  {
3  public:
4      explicit chronometre(std::string_view nom)
5          : nom_{nom}, debut_{std::chrono::steady_clock::now()}
6      {}
7      ~chronometre()
8      {
9          auto fin = std::chrono::steady_clock::now();
10         std::cout << nom_ << " : "
11                 << std::chrono::duration<double, std::milli>(fin - debut_).count()
12                 << " ms\n";
13     }
14 private:
15     std::string nom_;
16     std::chrono::time_point<std::chrono::steady_clock> debut_;
17 };
```

L'emploi dans une fonction

```
1 void traiter(std::vector<double>& v)
2 {
3     chronometre c{"traitement"};      // démarre la mesure
4
5     if (v.empty()) return;           // la mesure est affichée ici
6
7     for (auto& x : v) x = std::sqrt(x);
8                                     // ... ou bien ici
9 }
```

La propriété fondamentale

- Il **n'existe aucun** chemin de sortie qui oublie l'affichage
- Y compris si une exception traversait la boucle

La contrepartie

- Il faut penser en **termes de portées**, et parfois en créer explicitement

Envelopper une ressource du C

Un deleter pour un FILE du C

```
1 struct ferme_fichier
2 {
3     void operator()(std::FILE* f) const noexcept { if (f) std::fclose(f); }
4 };
5 using fichier = std::unique_ptr<std::FILE, ferme_fichier>;
6 fichier ouvrir(char const* chemin) { return fichier{std::fopen(chemin, "r")}; }
```

Bénéfices

- On ne **réécrit aucune des cinq fonctions membres spéciales**
- La fermeture est garantie, y compris si une exception traverse

Le détail qui compte

- Un deleter de type vide **ne coûte rien** en taille
- Une fonction libre ajouterait un pointeur

Construction en deux temps

L'objet incomplet comme anti-patron

- Il existe un état où l'objet est construit sans être valide
- Chaque fonction membre doit alors vérifier cet état
- L'invariant de classe n'est plus établi par le constructeur

```
1  connexion c;                // objet construit... mais inutilisable
2  c.init("serveur", 8080);    // et si on oublie cet appel ?
```

L'alternative sans exception

- Un constructeur ne peut signaler un échec qu'en levant
- Là où les exceptions sont proscrites — embarqué, jeu vidéo
- Un **constructeur nommé** renvoie un optional ou un expected

LA PROMESSE EST VÉRIFIÉE À L'EXÉCUTION

Si une exception tente de quitter une fonction noexcept, le programme appelle `std::terminate` : pas de propagation, pas de récupération.

Conséquences

- noexcept est **un engagement**, pas une optimisation à saupoudrer
- Un destructeur est noexcept par défaut : y lever termine le programme

Où le mettre vraiment

- Sur le **constructeur et l'affectation de déplacement** : la bibliothèque en dépend
- Sur swap et sur les accesseurs triviaux

shared_ptr : quand, et à quel prix

Coût

- Deux allocations si l'on écrit `shared_ptr<T>(new T)`, une seule avec `make_shared`
- Le **compteur est atomique** : chaque copie et chaque destruction se synchronisent

Le piège du cycle

- Deux objets qui se pointent **mutuellement ne sont jamais détruits**
- La sortie est `weak_ptr` sur l'un des deux sens

Le critère de choix

- `unique_ptr` par défaut : la propriété est presque toujours unique
- `shared_ptr` seulement si la durée de vie dépend de plusieurs détenteurs
- Et qu'aucune hiérarchie claire ne les ordonne

Tableau des responsabilités

Ressource	Outil
Mémoire, propriété unique	unique_ptr, ou un conteneur
Mémoire, propriété partagée	shared_ptr et weak_ptr
Ressource du C	unique_ptr avec <i>deleter</i>
Verrou	scoped_lock, unique_lock
Action de sortie quelconque	garde de portée

Règle générale

- On n'écrit **jamais delete** dans du code applicatif
- Toute ressource acquise l'est par un objet dont le destructeur la rend

Écrire une classe qui gère une ressource

La question à se poser d'abord

- Ai-je vraiment besoin de gérer la **ressource moi-même** ?
- Un `unique_ptr` ou un conteneur suffit dans la quasi-totalité des cas
- On retombe alors sur la **règle de zéro**

Si la réponse est oui

- Écrire les cinq fonctions membres spéciales, ou les supprimer
- Marquer déplacement et swap en `noexcept`
- Ne jamais laisser un constructeur sortir en ayant acquis à moitié

Le test qui ne trompe pas

- Deux objets, un swap, une exception au milieu : l'état reste-t-il cohérent ?

Un compte protégé par un verrou

```
1  class banque
2  {
3  public:
4  void transferer(compte& a, compte& b, long montant)
5  {
6      std::scoped_lock verrou{a.m_, b.m_};    // les deux, sans interblocage
7      a.retirer(montant); b.deposer(montant);
8  }
9  };
```

Bénéfices

- Déverrouillage garanti à la sortie de portée, exception comprise
- Verrouillage de plusieurs mutex dans un ordre évitant l'interblocage

L'ANTI-PATRON

Appeler `mutex::lock()` et `mutex::unlock()` à la main.

Niveaux de sûreté face aux exceptions

Niveau	Promesse si une exception passe	Prix
aucune	rien : fuite ou invariant rompu	à proscrire
de base	pas de fuite, invariants valides	quasi nul
forte	échec sans effet, tout ou rien	souvent une copie
noexcept	l'opération ne peut pas échouer	contraint le code

Le socle

- La garantie de base est **le minimum acceptable** : le code RAII l'obtient seul
- L'absence de garantie est un défaut, et non un quatrième niveau

Obtenir la garantie forte

La technique générale

- Faire **tout le travail susceptible d'échouer** sur une copie
- Ne modifier l'objet que par des opérations noexcept

```
1 void collection::ajouter_tous(std::vector<element> const& nouveaux)
2 {
3     auto copie = data_;           // peut lever : sans effet
4     for (auto const& e : nouveaux)
5         copie.push_back(e);       // peut lever : sans effet
6
7     data_.swap(copie);            // noexcept : point de validation
8 }
```

Le coût

- Une copie complète : ce n'est pas toujours acceptable
- La garantie forte est un **choix de conception**, pas un automatisme

Choisir un niveau

La règle par défaut

- Écrire du code RAII donne **la garantie de base** sans y penser
- La garantie forte se décide : elle ne s'obtient jamais par accident

Où noexcept est exigé

- Les destructeurs, qui le sont déjà par défaut
- Le **constructeur et l'affectation de déplacement**
- vector en dépend pour ne pas copier à la réallocation
- swap, sur lequel repose l'idiome copy and swap

Où la garantie forte se justifie

- Une opération que l'appelant peut vouloir réessayer telle quelle
- Une modification qui, échouant à mi-chemin, laisserait l'objet inutilisable

Garanties des conteneurs standards

Opération	Garantie
<code>vector::push_back</code>	forte : rien n'a changé si ça lève
<code>vector::insert</code> au milieu	de base seulement
<code>std::sort</code>	de base
swap de deux conteneurs	noexcept
Destruction	noexcept

La condition cachée

- `push_back` n'offre la garantie forte que si le déplacement est noexcept
- Sinon la **bibliothèque copie** pour rester sûre
- Un type déplaçable mais non noexcept fait recopier tout le tableau

Destructeurs et exceptions

La règle absolue

- Un **destructeur ne doit jamais** laisser échapper une exception
- Depuis C++11, il est implicitement noexcept
- S'il en laisse échapper une, `std::terminate` est appelé

Pourquoi

- Appelé pendant le déroulement de la pile, il ferait deux exceptions

```
1 ~connexion()  
2 {  
3     try { fermer(); }  
4     catch (...) { /* journaliser, ne surtout pas relancer */ }  
5 }
```

Conséquence de conception

- Si la libération peut échouer, offrir un `close()` explicite en plus

Un garde de portée générique

Un garde paramétré par une action

- Le garde exécute à la sortie de portée une action que `desamorcer` annule

```
1  template<typename F>
2  class garde
3  {
4  public:
5      explicit garde(F f) : f_{std::move(f)} {}
6      ~garde() { if (actif_) f_(); }
7      void desamorcer() noexcept { actif_ = false; }
8      garde(garde const&) = delete; garde& operator=(garde const&) = delete;
9  private:
10     F    f_;    bool actif_ = true;
11 };
```

Usage typique

```
1  auto* h = ouvrir_ressource();
2  garde g{[&]{ fermer_ressource(h); }};    // libere quoi qu'il arrive
```

Ce qu'il faut retenir — RAII

Principe

- Toute ressource est possédée par un objet
- La destruction déterministe garantit la libération

Les conséquences

- Le code de nettoyage n'apparaît **qu'une fois**, dans le destructeur
- Les chemins d'erreur deviennent triviaux
- Il n'y a plus de `goto cleanup` ni de `finally`

La règle pratique

- `new` et `delete` explicites n'apparaissent que dans une classe RAII dédiée
- Dans le code applicatif, **ils ne doivent jamais** apparaître

Mise en pratique : un verrou de répertoire

La ressource et sa durée de vie

- Le verrou est un fichier marqueur créé dans le dossier
- Il existe exactement le temps de la portée qui le déclare

```
1  class verrou_repertoire
2  {
3      public:
4          explicit verrou_repertoire(std::filesystem::path dossier)
5              : marque_{dossier / ".verrou"}
6          {
7              if (std::filesystem::exists(marque_))
8                  throw std::runtime_error{"repertoire deja verrouille"};
9
10             std::ofstream{marque_} << "1";
11         }
12
13         ~verrou_repertoire() { if (!marque_.empty()) std::filesystem::remove(marque_); }
14
15     private:
16         std::filesystem::path marque_;
17     };
```

Contraintes

- Deux objets détenant le même verrou le retireraient deux fois
- Le transfert reste légitime : un seul détenteur à la fois

```
1 verrou_repertoire(verrou_repertoire const&)          = delete;  
2 verrou_repertoire& operator=(verrou_repertoire const&) = delete;  
3  
4 verrou_repertoire(verrou_repertoire&& a) noexcept  
5     : marque_{std::exchange(a.marque_, {})} {}
```

```
verrou pose : 1  
second verrou refuse : repertoire deja verrouille  
verrou retire a la sortie de portee : 1
```

Notions employées

- L'acquisition est la construction : il n'y a pas d'objet sans verrou posé
- L'exception du constructeur ne laisse **aucun objet**, donc rien à nettoyer
- Le destructeur libère quel que soit le chemin de sortie, exception comprise
- La source vidée par exchange rend le destructeur du déplacé inoffensif

Subtilités

- Le destructeur ne lève pas : remove échouant, il n'y a rien à signaler
- Un `verrou.liberer()` explicite rouvrirait la fenêtre que RAII vient de fermer
- Le test puis la création ne sont pas atomiques : l'exercice porte sur RAII

7. Principes de conception et SOLID

KISS — Keep It Simple, Stupid

- La solution la plus simple qui résout le problème est la bonne
- La **complexité doit être justifiée**, pas subie

DRY — Don't Repeat Yourself

- Toute connaissance doit avoir une **représentation unique** dans le système
- Attention : deux codes identiques par coïncidence ne sont pas une duplication

YAGNI — You Aren't Gonna Need It

- Ne pas implémenter ce dont on n'a pas **besoin aujourd'hui**
- L'extensibilité spéculative est la première source de complexité inutile

Énoncé

- Une fonction membre de C ne devrait appeler que les fonctions de :
 - C elle-même
 - ses propres membres
 - ses paramètres
 - les objets qu'elle crée

Formulation informelle

- « Ne parlez qu'à vos amis immédiats »

Une chaîne d'appels et sa correction

Avant, et après

```
1  // Violation : on traverse trois niveaux d'abstraction  
2  commande.client().adresse().ville().code_postal();  
3  
4  // Respect : on demande Le service, pas Les données  
5  commande.code_postal_livraison();
```

Bénéfices

- Le code appelant ne dépend plus de la structure interne de commande

Le couplage

- Mesure la dépendance entre deux modules
- Un couplage fort signifie qu'un changement se propage

La cohésion

- Mesure à quel point les éléments d'un module vont ensemble
- Une cohésion forte signifie que le module a un but unique

L'objectif (Larry Constantine, 1974)

- Un **couplage faible** et une **cohésion forte**
- C'est la formulation la plus ancienne et la plus générale de toutes

Degrés de couplage

Du plus faible au plus fort

- **Par données** : on échange des valeurs simples — le meilleur cas
- **Par structure** : on échange un agrégat dont on n'utilise qu'une partie
- **Par contrôle** : un argument pilote le comportement de l'appelé
 - Un bool en paramètre est presque toujours ce cas
- **Par variable globale** : deux modules partagent un état mutable
- **Par contenu** : un module manipule l'interne d'un autre — le pire cas

En C++

- L'héritage public est un couplage très fort
- Une dépendance d'en-tête est déjà un couplage de compilation

Du plus faible au plus fort

- **Fortuite** : les éléments sont regroupés par hasard — une classe `Utils`
- **Logique** : regroupés parce qu'ils font des choses « du même genre »
- **Temporelle** : regroupés parce qu'ils s'exécutent au même moment
- **Procédurale** : regroupés parce qu'ils s'enchaînent
- **Communicationnelle** : ils opèrent sur les mêmes données
- **Fonctionnelle** : ils concourent tous à une **unique** tâche bien définie

Le test simple

- Peut-on décrire la classe en une phrase, sans « et » ?
- Sinon, la cohésion est probablement trop faible

Cinq principes formulés par Robert C. Martin

- **S** — *Single Responsibility Principle*
- **O** — *Open/Closed Principle*
- **L** — *Liskov Substitution Principle*
- **I** — *Interface Segregation Principle*
- **D** — *Dependency Inversion Principle*

Leur statut

- Ce sont des **heuristiques, pas des lois**
- Elles décrivent toutes une facette de « couplage faible, cohésion forte »
- Elles s'appliquent à l'orienté objet, mais se transposent au générique

S — Responsabilité unique

Énoncé

- Une classe ne doit avoir **qu'une seule raison de changer**

Reformulation utile

- Une classe doit répondre à un **seul groupe d'acteurs**
- Comptabilité et technique demandent des changements dans la même classe
- Elle porte alors deux responsabilités

Bénéfices

- Les changements restent localisés
- Les tests deviennent simples et rapides
- La réutilisation devient possible

Une classe à trois responsabilités

```
1  class employe
2  {
3  public:
4      double calculer_paie() const;           // regle metier    -> comptabilite
5      void    sauvegarder() const;           // persistance      -> DBA
6      std::string vers_html() const;         // presentation     -> equipe web
7      void    envoyer_courriel() const;      // notification     -> equipe infra
8  };
```

Le diagnostic

- Quatre acteurs distincts peuvent demander un changement
- Toute modification impose de recompiler et retester l'ensemble
- La classe dépend de la base de données, du HTML et du réseau

S — Conception corrigée

Une classe par responsabilité

```
1 // Donnees et regles metier pures
2 class employe
3 {
4     public:
5     double salaire_base() const;
6     int    heures_supplementaires() const;
7     private:
8     // ...
9 };
10
11 // Chaque responsabilite dans son propre composant
12 class calculateur_paie { public: double calculer(employe const&) const; };
13 class depot_employe   { public: void sauvegarder(employe const&); };
14 class formateur_html  { public: std::string formater(employe const&) const; };
15 class service_courriel { public: void envoyer(employe const&, std::string_view); };
```

Le gain

- Changer le format HTML n'impose plus de retester le calcul de paie

Énoncé

- Un module doit être **ouvert à l'extension** et **fermé à la modification**

Traduction

- On doit pouvoir ajouter un comportement **sans toucher au code existant**
- Le code existant reste testé et validé

Les mécanismes en C++

- Polymorphisme dynamique par interface
- Surcharge de fonctions et ADL
- Templates et spécialisation
- Injection de comportement par fonction ou lambda

Une fonction à modifier à chaque ajout

```
1  enum class type_forme { cercle, rectangle, triangle };
2
3  double aire(forme_brute const& f)
4  {
5      switch (f.type)
6      {
7          case type_forme::cercle:    return 3.14159 * f.r * f.r;
8          case type_forme::rectangle: return f.l * f.h;
9          case type_forme::triangle:  return 0.5 * f.b * f.h;
10     }
11     return 0.0;                // sinon -Wreturn-type
12 }
```

Le problème

- Ajouter un losange impose de modifier aire
- ... et perimetre, et dessiner, et toutes les autres fonctions
- Chaque ajout risque d'introduire une régression ailleurs

O — Conception corrigée

L'ajout par dérivation

```
1  class forme
2  {
3      public:
4          virtual ~forme() = default;
5          virtual double aire() const = 0;
6  };
7  class losange : public forme    // ajout SANS toucher au code existant
8  {
9      public:
10         losange(double d1, double d2) : d1_{d1}, d2_{d2} {}
11         double aire() const override { return d1_ * d2_ / 2.0; }
12     private:
13         double d1_, d2_;
14     };
```

La nuance importante

- OCP **facilite l'ajout de types**, mais **complique celui des opérations**
- Le switch fait exactement l'inverse : c'est le *problème d'expression*

Problème d'expression

Le constat

- Aucune approche ne permet d'ajouter facilement les deux à la fois

Approche	Ajouter un type	Ajouter une opération
Hierarchie virtuelle	facile	difficile
switch sur un enum	difficile	facile
<code>std::variant</code>	difficile	facile
Patron <i>Visitor</i>	difficile	facile

La décision de conception

- Quel **axe est ouvert** dans le domaine considéré ?
- Beaucoup de types, peu d'opérations → hiérarchie
- Peu de types, beaucoup d'opérations → `variant` ou `visitor`

Les signaux d'alarme

- Une redéfinition qui lève `std::logic_error` « non supporté »
- Une redéfinition qui **ne fait rien** alors que la base fait quelque chose
- Un `dynamic_cast` dans le code client pour distinguer les cas
- Une documentation qui dit « sauf pour la classe X »

Diagnostic

- C'est la **relation d'héritage qui est en cause**, pas la classe dérivée
- La correction porte sur la hiérarchie, jamais sur le code client

Une interface honnête

```
1  // interface minimale, honnête
2  class collection_lecture
3  {
4      public:
5          virtual ~collection_lecture() = default;
6          virtual std::size_t taille() const = 0;
7          virtual int          element(std::size_t) const = 0;
8  };
9
10 class collection_mutable : public collection_lecture
11 {
12     public:
13         virtual void ajouter(int) = 0;
14 };
```

Le principe appliqué

- Une collection immuable ne dérive pas d'une mutable en désactivant ajouter
- On sépare les interfaces **selon les capacités réelles**

I — Ségrégation des interfaces

Énoncé

- Aucun client ne doit dépendre de méthodes qu'il n'utilise pas

La conséquence

- Préférer plusieurs interfaces **petites et cohérentes** à une grosse
- Chaque client dépend uniquement de ce dont il a besoin

```
1  // Mise en défaut : une interface fourre-tout
2  class peripherique
3  {
4      public:
5          virtual void imprimer(document const&) = 0;
6          virtual void scanner(document&)        = 0;
7          virtual void faxer(document const&)     = 0;
8  };
9
10 // Une imprimante simple doit implementer scanner() et faxer()
11 // ... en levant une exception : violation de LSP par ricochet
```

I — Conception corrigée

Des interfaces séparées

```
1  class imprimante { public: virtual ~imprimante() = default;
2                          virtual void imprimer(document const&) = 0; };
3  class scanner      { public: virtual ~scanner() = default;
4                          virtual void scanner_vers(document&) = 0; };
5
6  class telecopieur{ public: virtual ~telecopieur() = default;
7                          virtual void faxer(document const&) = 0; };
8
9  // Une imprimante simple
10 class laserjet : public imprimante { /* ... */ };
11
12 // Un multifonction compose les capacites reellement offertes
13 class multifonction : public imprimante, public scanner, public telecopieur
14 { /* ... */ };
```

Bénéfices

- Le code qui imprime ne dépend plus du télécopieur
- Chaque type n'implémente que ce qu'il sait réellement faire

D — Inversion des dépendances

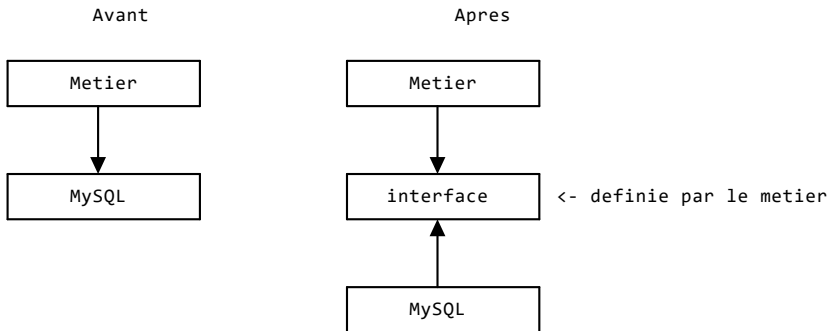
Énoncé

- Les modules de haut niveau ne doivent pas dépendre des modules de bas niveau
- Les deux doivent **dépendre d'abstractions**
- Les abstractions ne doivent pas dépendre des détails

Le point essentiel

- L'abstraction appartient au **module de haut** niveau

D — Le renversement en image



Une dépendance vers le concret

```
1  #include "mysql_connexion.hpp"    // dependance concrete
2
3  class service_commande
4  {
5      public:
6      void enregistrer(commande const& c)
7      {
8          mysql_.executer("INSERT INTO commandes ...");
9      }
10
11     private:
12     mysql_connexion mysql_;        // couplage fort et definitif
13 };
```

Les conséquences

- Impossible de tester sans une base MySQL disponible
- Impossible de changer de moteur sans réécrire le métier
- Le temps de compilation dépend de l'en-tête MySQL

L'abstraction posée par le métier

```
1  class depot_commande
2  {
3      public:
4          virtual ~depot_commande() = default;
5          virtual void enregistrer(commande const&) = 0;
6  };
7  class service_commande
8  {
9      public:
10         explicit service_commande(depot_commande& d) : depot_{d} {}    // injection
11         void traiter(commande const& c) { depot_.enregistrer(c); }
12     private:
13         depot_commande& depot_;
14     };
```

Bénéfices

- Un depot_memoire permet de tester sans base de données
- Le moteur de persistance devient un détail interchangeable

Injection de dépendances en C++ moderne

Trois formes possibles

```
1  // 1. Interface virtuelle : polymorphisme a l'execution
2  class service { public: explicit service(depot& d); };
3
4  // 2. Parametre template : polymorphisme a la compilation, cout nul
5  template<typename Depot>
6  class service { public: explicit service(Depot& d); };
7
8  // 3. std::function : injection d'un comportement, pas d'un objet
9  class service
10 {
11     public: explicit service(std::function<void(commande const&)> enregistrer);
12 };
```

Choisir

- L'implémentation change-t-elle à l'exécution ? → interface virtuelle
- Est-elle fixée à la compilation ? → template, sans coût
- S'agit-il d'un seul point de variation ? → `std::function`

Ce qu'il faut retenir — Principes

Le socle

- Couplage faible, cohésion forte
- Tout le reste en découle

SOLID

- **S** : une seule raison de changer
- **O** : étendre sans modifier
- **L** : les sous-types respectent les contrats
- **I** : des interfaces petites et ciblées
- **D** : dépendre d'abstractions que l'on possède

La mise en garde

- Ces principes **ont un coût en indirections et en abstractions**
- Les appliquer aveuglément produit du code sur-conçu
- Ils répondent à un besoin de **changement avéré**, pas supposé

Mise en pratique : un exportateur de relevés

Deux abstractions posées par le métier

- Ce qui mettra en forme, et ce qui recevra le résultat
- L'exportateur ne connaît ni le format concret ni la destination concrète

```
1  class formateur
2  { public: virtual ~formateur() = default;
3          virtual std::string mettre_en_forme(releve const&) const = 0; };
4
5  class destination
6  { public: virtual ~destination() = default;
7          virtual void ecrire(std::string const&) = 0; };
8
9  class exportateur
10 {
11     public:
12     exportateur(formateur const& f, destination& d) : format_{f}, sortie_{d} {}
13     void exporter(std::vector<releve> const& releves);
14     private:
15     formateur const& format_;
16     destination&     sortie_;
17 };
```

Notions employées

- **Responsabilité unique** : mettre en forme, écrire et orchestrer sont trois classes
- **Ouvert / fermé** : un format de plus ne modifie aucune ligne existante
- **Inversion des dépendances** : les deux interfaces appartiennent au métier
- L'injection se fait par le constructeur, sans fabrique ni configuration

Subtilités

- Une destination en mémoire suffit à tester : aucun fichier, aucun réseau
- C'est le signe que l'inversion est réelle et non décorative

```
tension;3.30  
intensite;0.75
```

8. Patrons de conception

Qu'est-ce qu'un patron de conception ?

Définition

- **Une solution éprouvée à un problème de conception récurrent**
- Décrite indépendamment d'un langage particulier

Bénéfices

- **Un vocabulaire commun** entre développeurs, et les compromis de chacun

Limitations

- Ce n'est pas une bibliothèque de recettes à appliquer
- Ni une preuve de la qualité d'un code

Le contexte historique

- *Design Patterns* (Gamma, Helm, Johnson, Vlissides — 1994)
- Beaucoup de patrons **compensent des manques du C++ de l'époque**

The Sacred Elements of the Faith

the holy
origins

the holy
structures

107 FM Factory Method							139 A Adapter
117 PT Prototype	127 S Singleton				223 CR Chain of Responsibility	163 CP Composite	175 D Decorator
87 AF Abstract Factory	325 TM Template Method	233 CD Command	273 MD Mediator	293 O Observer	243 IN Interpreter	207 PX Proxy	185 FA Façade
97 BU Builder	315 SR Strategy	283 MM Memento	305 ST State	257 IT Iterator	331 V Visitor	195 FL Flyweight	151 BR Bridge

- **Créationnels** : contrôler la construction des objets
- **Structurels** : composer objets et classes
- **Comportementaux** : répartir les responsabilités et les algorithmes

Objectifs

- Découpler le client du type concret qu'il instancie
- Contrôler le moment, le lieu et la manière de la construction

Les principaux

- **Factory Method** : une fonction virtuelle décide du type créé
- **Abstract Factory** : une famille cohérente de produits liés
- **Builder** : construire un objet complexe par étapes
- **Prototype** : créer par clonage d'un exemplaire existant
- **Singleton** : garantir une instance unique — à manier avec précaution

Factory : mise en œuvre

Une fonction de création

```
1  class forme { public: virtual ~forme() = default;
2                      virtual double aire() const = 0; };
3
4  std::unique_ptr<forme> creer_forme(std::string_view type,
5                                   std::span<double const> params)
6  {
7      if (type == "cercle")    return std::make_unique<cercle>(params[0]);
8      if (type == "rectangle") return std::make_unique<rectangle>(params[0], params[1]);
9      throw std::invalid_argument{"type de forme inconnu"};
10 }
```

Bénéfices

- Le client ne connaît que forme et le nom du type
- La **table de correspondance** est centralisée en un seul point
- Elle peut devenir dynamique : enregistrement de types à l'exécution

Une construction par étapes

- Un objet à nombreux paramètres optionnels rend le constructeur illisible
- Le constructeur intermédiaire nomme chaque étape et valide à la fin

```
1  class requete_http
2  {
3      public:
4          class constructeur
5          {
6              public:
7                  constructeur& methode(std::string m) { m_ = std::move(m); return *this; }
8                  constructeur& url(std::string u)      { u_ = std::move(u); return *this; }
9                  constructeur& entete(std::string k, std::string v);
10                 requete_http construire() &&;    // valide les invariants ici
11             };
12     };
13
14     auto r = requete_http::constructeur{
15         .methode("GET").url("/api/v1").entete("Accept", "json")
16         .construire();
```

Singleton : le patron controversé

L'instance unique

```
1  class journal
2  {
3      public:
4          static journal& instance()
5              { static journal j; return j; }      // initialisation thread-safe
6          journal(journal const&)                = delete;
7          journal& operator=(journal const&)      = delete;
8      private:
9          journal() = default;
10 };
```

Les problèmes réels

- C'est une **variable globale** déguisée : couplage invisible
- Rend les tests difficiles : l'état est partagé entre eux

L'ALTERNATIVE

Créer une instance unique dans `main` et l'injecter explicitement.

Objectifs

- Composer des objets pour former des structures plus riches
- Adapter, enrichir ou masquer une interface existante

Les principaux

- **Adapter** : convertir une interface existante vers celle attendue
- **Decorator** : ajouter des responsabilités à interface constante
- **Composite** : traiter uniformément un objet et un groupe d'objets
- **Facade** : offrir une interface simple sur un sous-système complexe
- **Proxy** : contrôler l'accès à un objet — cache, droits, chargement différé
- **Bridge** / **PIMPL** : séparer l'abstraction de son implémentation

Decorator : mise en œuvre

Une décoration empilable

- Le décorateur implémente l'interface qu'il enrichit et délègue au suivant
- Le client ne voit qu'un flux, quelle que soit la pile

```
1  class flux { public: virtual ~flux() = default;
2                      virtual void ecrire(std::string_view) = 0; };
3  class flux_fichier : public flux { /* ... */ };
4  class flux_compresse : public flux      // decorateur
5  {
6  public:
7      explicit flux_compresse(std::unique_ptr<flux> suivant)
8          : suivant_{std::move(suivant)} {}
9
10     void ecrire(std::string_view d) override
11     {
12         suivant_->ecrire(compresser(d));    // enrichit puis delegue
13     }
14
15     private:
16         std::unique_ptr<flux> suivant_;
17 };
```

Composition à l'exécution

- Les décorateurs se combinent dans l'ordre voulu, à l'exécution
- Chaque couche ignore ce qu'il y a au-dessus et au-dessous

```
1  auto f = std::make_unique<flux_compresse>(
2      std::make_unique<flux_chiffre>(
3      std::make_unique<flux_fichier>("sortie.dat")));
```

PIMPL : l'idiome de compilation

L'en-tête sans détail d'implémentation

- L'en-tête ne déclare qu'un pointeur vers une implémentation opaque
- L'en-tête ne cite aucun type de l'implémentation

```
1  // widget.hpp -- aucun en-tete d'implementation expose
2  class widget
3  {
4      public:
5          widget();
6          ~widget();                      // DOIT etre defini dans le .cpp
7          widget(widget&&) noexcept;
8          widget& operator=(widget&&) noexcept;
9
10     void dessiner() const;
11
12     private:
13         struct impl;
14         std::unique_ptr<impl> pimpl_;
15     };
```

Bénéfices

- L'ABI reste stable quand l'implémentation change
- Les clients ne recompilent pas quand `impl` change
- Les dépendances lourdes restent confinées au `.cpp`

Le prix

- Une indirection et une allocation dynamique par objet

Objectifs

- Répartir les responsabilités entre objets
- Rendre les algorithmes interchangeables à l'exécution

Les principaux

- **Strategy** : encapsuler un algorithme interchangeable
- **Observer** : notifier des dépendants d'un changement d'état
- **Visitor** : ajouter des opérations sans modifier les types
- **Template Method** : fixer le squelette, laisser varier les étapes
- **State** : le comportement dépend d'un état interne
- **Command** : réifier une action en objet — annulation, file d'attente

Strategy : version classique

Une stratégie par héritage

- L'algorithme devient un objet, choisi par le client
- Le trieur ne connaît que l'interface, jamais l'algorithme concret

```
1  class strategie_tri
2  {
3      public:
4          virtual ~strategie_tri() = default;
5          virtual void trier(std::span<int>) const = 0;
6  };
7  class tri_rapide : public strategie_tri { /* ... */ };
8  class tri_fusion : public strategie_tri { /* ... */ };
9  class trieur
10 {
11     public:
12         explicit trieur(std::unique_ptr<strategie_tri> s) : s_{std::move(s)} {}
13         void trier(std::span<int> d) const { s_->trier(d); }
14
15     private:
16         std::unique_ptr<strategie_tri> s_;
17 }
```

Strategy : versions modernes

Avec `std::function` — plus léger

- La stratégie n'exige plus de hiérarchie : tout callable convient

```
1  class trieur
2  {
3      public:
4          explicit trieur(std::function<void(std::span<int>)> s) : s_{std::move(s)} {}
5          void trier(std::span<int> d) const { s_(d); }
6      private:
7          std::function<void(std::span<int>)> s_;
8  };
9
10  trieur t{[] (std::span<int> d) { std::ranges::sort(d); }};
```

Avec un paramètre template — coût nul

Stratégie fixée à la compilation

La stratégie en paramètre de type

```
1  template<typename Strategie>
2  class trieur
3  {
4      public:
5          explicit trieur(Strategie s) : s_{std::move(s)} {}
6          void trier(std::span<int> d) const { s_(d); }
7      private:
8          Strategie s_;
9  };
```

Le critère de choix

- La stratégie change-t-elle à l'exécution ? Sinon, le template suffit

Le sujet et ses observateurs

- Le sujet tient une liste d'observateurs et les notifie à chaque changement
- Il ne connaît que l'interface d'observation, jamais les types concrets

```
1  class sujet
2  {
3      public:
4          using rappel = std::function<void(int)>;
5          std::size_t abonner(rappel r)
6          {
7              observateurs_.push_back(std::move(r));
8              return observateurs_.size() - 1;
9          }
10         void notifier(int valeur) const
11         {
12             for (auto const& o : observateurs_) o(valeur);
13         }
14
15     private:
16         std::vector<rappel> observateurs_;
17     };
```

Les pièges classiques

- Durée de vie : un observateur détruit mais toujours abonné
- Ordre de notification non spécifié
- Modification de la **liste pendant** la notification

La parade

- Désabonnement explicite dans le destructeur de l'observateur
- Copie de la liste avant de notifier, pour tolérer les modifications

Du Visitor classique au variant

Le patron Visitor classique

- Deux hiérarchies, double dispatch par `accept` et `visit`
- Verbeux et intrusif : chaque type doit connaître le visiteur

La version moderne

- `std::variant` tient les types, `std::visit` porte l'opération
- Aucune hiérarchie, aucun `accept` à écrire dans les types

Un variant de formes

```
1  struct cercle    { double r; };
2  struct rectangle { double l, h; };
3  struct triangle  { double b, h; };
4
5  using forme = std::variant<cercle, rectangle, triangle>;
6
7  double aire(forme const& f)
8  {
9      return std::visit(overloaded{
10         [](cercle const& c)    { return 3.14159265 * c.r * c.r; },
11         [](rectangle const& r) { return r.l * r.h; },
12         [](triangle const& t)  { return 0.5 * t.b * t.h; },
13     }, f);
14 }
```

Les bénéfices

- Aucune allocation dynamique, aucun pointeur, aucune vtable
- Ajouter une opération est trivial ; ajouter un type impose de tout mettre à jour

Assistant overloaded

La définition de l'assistant

```
1  template<typename... Ts>
2  struct overloaded : Ts...
3  {
4      using Ts::operator()...;
5  };
6
7  // Guide de deduction, superflu depuis C++20
8  template<typename... Ts> overloaded(Ts...) -> overloaded<Ts...>;
```

Comment cela fonctionne

- Héritage multiple de toutes les lambdas passées
- `using Ts::operator()...;` amène tous les `operator()` dans la même surcharge
- La résolution de surcharge choisit la bonne au moment de la visite

Statut

- C'est l'un des idiomes les plus utiles du C++ moderne

Patrons devenus obsolètes

Rendus superflus par le langage

- **Iterator** : les itérateurs et les *ranges* sont dans la bibliothèque
- **Command** : une lambda ou `std::function` suffit
- **Strategy** : idem, ou un paramètre template
- **Prototype** : le constructeur de copie remplit le rôle
- **Template Method** : souvent remplaçable par une injection de fonction

La leçon

- Beaucoup de patrons GoF sont des **contournements de limitations**
- Ceux qui restent essentiels décrivent une **véritable structure**
 - Composite, Decorator, Observer, Facade, Bridge

Mise en pratique : une machine à pile

Un jeu d'instructions fermé

- Trois instructions, connues à l'écriture du programme
- Chacune est un type, l'ensemble est un `std::variant`

```
1 struct empiler    { double valeur; };
2 struct additionner {};
3 struct multiplier  {};
4
5 using instruction = std::variant<empiler, additionner, multiplier>;
6
7 for (instruction const& i : programme)
8     std::visit(overloaded{
9         [&](empiler const& e)    { pile.push_back(e.valeur);          },
10        [&](additionner const&) { binaire(std::plus<double>{});        },
11        [&](multiplier const&)  { binaire(std::multiplies<double>{}); }
12    }, i);
```

Notions employées

- **C'est Visitor**, sans hiérarchie, sans accept et sans double distribution
- overloaded fabrique le visiteur à partir de trois fonctions anonymes
- Ajouter une opération sur le jeu d'instructions ne touche à aucun type
- Aucune allocation, aucune table virtuelle : tout tient dans le variant

Subtilités

- Le compilateur exige un cas **par type** : oublier une instruction ne compile pas
- Ajouter une instruction impose en revanche de reprendre chaque visiteur

```
empiler 2, empiler 3, additionner, empiler 4, multiplier -> 20
```

9. Au-delà du polymorphisme dynamique

Coût

- Un pointeur supplémentaire par objet
- Deux indirections mémoire par appel
- Une barrière à l'*inlining* et donc à toutes les optimisations qui suivent
- Un saut indirect potentiellement mal prédit

Le vrai coût

- Ce n'est pas **l'indirection en elle-même**
- C'est l'**inlining perdu** : trois lignes non *inlinées* bloquent tout le reste
- Propagation de constantes, vectorisation, déroulage

Quand cela devient un problème

- Dans une boucle chaude, sur des millions d'appels
- Jamais dans du code de configuration ou d'orchestration

Un appel dont le type est connu

```
1 struct base { virtual void f() const; };
2 struct derivee final : base { void f() const override; };
3
4 void g(derivee const& d) { d.f(); } // appel direct : le type est connu
5 void h(base const& b)    { b.f(); } // appel indirect, sauf si le
6                               // compilateur voit le type réel
```

La dévirtualisation

- Si le type dynamique est connu au point d'appel, l'appel virtuel disparaît
- `final` sur la classe ou la fonction aide le compilateur à le prouver
- L'optimisation à l'édition de liens élargit ce qu'il peut voir
- Une collection de pointeurs de base y échappe : le type varie

Analyser le besoin réel

Les questions à se poser

- L'ensemble des **types est-il fermé** et connu à la compilation ?
- La variation doit-elle être **choisie à l'exécution** ?
- Types dérivés **de tiers** ? Point de variation en **chemin critique** ?

Les alternatives selon les réponses

Situation	Solution
Types fermés, choix à la compilation	Templates, surcharge
Types fermés, choix à l'exécution	<code>std::variant</code>
Types ouverts, choix à l'exécution	Virtuel ou effacement de type
Réutilisation d'implémentation statique	CRTP
Types forts sans polymorphisme	Types opaques

Alternative : la surcharge et les templates

Une surcharge par type

```
1  struct cercle    { double r; };
2  struct rectangle { double l, h; };
3
4  double aire(cercle const& c)    { return 3.14159265 * c.r * c.r; }
5  double aire(rectangle const& r) { return r.l * r.h;                }
6
7  template<typename Forme>
8  double aire_totale(std::span<Forme const> formes)
9  {
10     double t = 0.0;
11     for (auto const& f : formes) t += aire(f);    // resolu a la compilation
12     return t;
13 }
```

À considérer si

- Le type est connu au site d'appel
- Le code doit être *inliné* et vectorisé

Alternative : std::variant

Une collection hétérogène

```
1  using forme = std::variant<cercle, rectangle, triangle>;
2
3  std::vector<forme> formes;           // collection heterogene, sans allocation
4  formes.push_back(cercle{1.0});
5  formes.push_back(rectangle{2.0, 3.0});
6
7  double total = 0.0;
8  for (auto const& f : formes)
9      total += std::visit([](auto const& x) { return aire(x); }, f);
```

À considérer si

- L'ensemble des **types est fermé** et connu
- On veut une collection hétérogène sans allocation ni indirection

Le coût

- Taille du plus grand membre plus un discriminant

La base paramétrée par la dérivée

- La base reçoit le type dérivé en paramètre et l'appelle directement
- Le polymorphisme est résolu à la compilation : aucun appel indirect

```
1  template<typename Derive>
2  class forme_base
3  {
4      public:
5          double aire() const { return static_cast<Derive const*>(*this).aire_impl(); }
6  };
7
8  class cercle : public forme_base<cercle>
9  {
10     public:
11         explicit cercle(double r) : r_{r} {}
12         double aire_impl() const { return 3.14159265 * r_ * r_; }
13     private:
14         double r_;
15 };
```

Du CRTP à l'effacement de type

Quand employer le CRTP

- On veut **factoriser du code de base** sans coût d'exécution
- Aucune collection hétérogène n'est nécessaire

Principe de l'effacement de type

- Offrir une interface **polymorphe à l'exécution** sans héritage imposé
- L'exemple canonique est `std::function`

Enveloppe d'effacement de type

Une enveloppe sans contrainte d'héritage

- Le type rangé n'a rien à dériver : l'enveloppe fabrique l'indirection

```
1  // type-erasure : aucune contrainte sur T
2  class forme
3  {
4      public:
5          template<typename T>
6          forme(T x) : impl_{std::make_unique<modele<T>>(std::move(x))} {}
7          double aire() const { return impl_->aire(); }
8      private:
9          struct concept_ { virtual ~concept_() = default; virtual double aire() const = 0; };
10         template<typename T>
11         struct modele : concept_
12         {
13             explicit modele(T x) : x_{std::move(x)} {}
14             double aire() const override { return ::aire(x_); } // la fonction libre, pas le membre
15             T x_;
16         };
17         std::unique_ptr<concept_> impl_;
18     };
```

Effacement de type : évaluation

Bénéfices

- Les types concrets **restent indépendants de la hiérarchie**
- **Sémantique de valeur** : on peut copier et stocker forme directement
- L'interface est définie par une **fonction libre**, pas par un héritage

Coût

- Une allocation dynamique et une indirection, comme le virtuel
- Un code d'infrastructure verbeux à écrire une fois par interface

Où on le rencontre

- `std::function`, `std::any`, `std::pmr::memory_resource`
- `std::move_only_function` (C++23)

std::function, l'effacement de type standard

Deux affectations de nature différente

```
1 std::function<int(int)> f = [](int x) { return x + 1; };  
2 std::function<int(int)> g = &une_fonction_libre;
```

Bénéfices

- N'importe quel appellable de la bonne signature, sans hiérarchie commune
- C'est exactement l'effacement de type, offert par la bibliothèque

Coût

- Un appel indirect, comme une fonction virtuelle
- **Une allocation** si l'appellable ne tient pas dans le tampon interne

L'alternative sans coût

- Un paramètre template appellable, quand le type est connu à la compilation

Mesurer avant de choisir

L'ordre de grandeur, sur une boucle serrée

Mécanisme	Coût relatif par appel
Appel direct, inséré en ligne	référence
Appel direct, non inséré	2 à 5 fois
Appel virtuel effectif	5 à 20 fois
<code>std::function</code>	5 à 25 fois

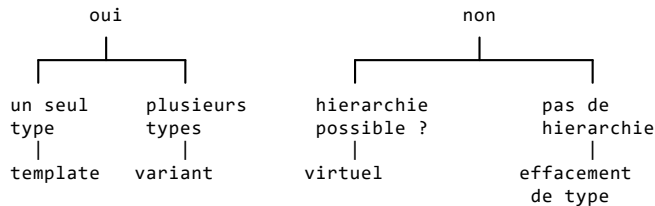
- Un appel **virtuel dévirtualisé** revient au coût d'un appel direct

Pourquoi la fourchette est si large

- Le coût ne vient pas de l'indirection mais de **l'inlining qu'elle empêche**
- Mesurable par élément de tableau, indétectable par requête réseau

Critère de décision

L'ensemble des types est-il connu a la compilation ?



La question préalable

- Ai-je vraiment plusieurs types, ou un seul avec des variantes de comportement ?
- Beaucoup de hiérarchies naissent d'un `if` que l'on a voulu déguiser

Trois erreurs fréquentes

Rendre virtuel « au cas où »

- Une fonction virtuelle est un **point d'extension public**
- Elle engage la classe autant qu'une fonction publique
- On ne rend virtuel que ce que l'on a décidé de laisser redéfinir

Hériter pour réutiliser du code

- L'héritage exprime « est un », pas « utilise »
- La composition ou un membre privé y suffisent sans exposer la base

Choisir avant de mesurer

- L'effacement de type et le CRTP compliquent le code
- On ne les paie qu'après un profil : le mécanisme simple coûte trop cher

Tableau de synthèse

Technique	Types ouverts	Choix exécution	Coût	Valeur
Fonctions virtuelles	oui	oui	moyen	non
<code>std::variant</code>	non	oui	faible	oui
Templates / surcharge	oui	non	nul	oui
CRTP	oui	non	nul	oui
Effacement de type	oui	oui	moyen	oui

Comment lire ce tableau

- « Valeur » : le type peut-il être manipulé par valeur, copié, stocké ?
- Aucune ligne n'est meilleure que les autres dans l'absolu
- Le choix se fait selon les contraintes réelles du problème

Mise en pratique : un journal, deux régimes

La même fonction, deux besoins

- Le format est-il connu à l'écriture du programme, ou choisi à l'exécution ?

```
1  template<typename Format>           // format fige a la compilation
2  class journal_statique
3  {
4      public:
5      explicit journal_statique(Format f) : format_{std::move(f)} {}
6      std::string ecrire(std::string_view m) const { return format_(m); }
7      private:
8      Format format_;
9  };
10 // format remplaçable a l'execution
11 class journal_dynamique
12 {
13     public:
14     using format = std::function<std::string(std::string_view)>;
15     void changer(format f) { format_ = std::move(f); }
16     private:
17     format format_;
18 };
```

Notions employées

- Le **patron inline** l'appel : le format disparaît du code engendré
- `std::function` garde un appel indirect et peut allouer, mais accepte tout
- Deux `journal_statique` de formats différents sont **deux types sans lien**
- Un seul `journal_dynamique` traverse les deux formats sans changer de type

Le critère

- Le format change à l'exécution, ou la collection est hétérogène : indirection
- Ni l'un ni l'autre : le paramètre de type, et rien ne subsiste à l'exécution

```
statique : démarrage  
dynamique : démarrage  
dynamique apres changement : [info] démarrage
```

Ce qu'il faut retenir — Synthèse générale

L'objet

- Une classe existe pour **protéger un invariant**
- L'encapsulation protège l'invariant, pas les données pour elles-mêmes
- L'héritage exprime une substituabilité, pas une réutilisation

La durée de vie

- C++ est un **langage de valeurs** avec destruction déterministe
- RAII transforme cette propriété en outil de gestion de ressources
- La règle de zéro est l'objectif ; la règle de cinq est le recours

La conception

- Couplage faible et cohésion forte, dont SOLID est une déclinaison
- Les patrons sont un vocabulaire, pas une méthode
- Le polymorphisme **dynamique est une** solution parmi plusieurs

Ouvrages

- Scott Meyers, *Effective Modern C++*
- Herb Sutter, *Exceptional C++* et *More Exceptional C++*
- Klaus Iglberger, *C++ Software Design*
- Gamma et al., *Design Patterns*
- John Lakos, *Large-Scale C++ Software Design*

Ressources en ligne

- *C++ Core Guidelines* — Stroustrup et Sutter
- cppreference.com
- Les conférences CppCon, Meeting C++, ACCU

Merci pour votre attention